# High-Performance Big Data Analytics Using R

## NYC Yellow Taxi Trip Records (January – June 2024)

**Laboratory Assignment 8 – R Programming**

**Author:** Swaroop Naik (@Swaroop192005)

---

## Contents

1. Problem Statement
2. Dataset
3. Task 1 – Data Acquisition and Preparation
4. Task 2 – Transportation Data Analytics
5. Task 3 – Functional vs Vectorized vs data.table Programming
6. Task 4 – Parallel Computing (foreach + doParallel)
7. Task 5 – Performance Benchmarking and Optimization
8. Task 6 – Data Visualization (ggplot2 + leaflet)
9. Observations
10. Task 7 – Critical Analysis and Recommendation
11. Conclusion

# 1. Problem Statement

The New York City Taxi and Limousine Commission (NYC TLC) publishes millions of taxi-trip records every month. Each record contains pickup/drop-off timestamps and locations, passenger count, trip distance, fare, tip, tolls, payment type and the total amount charged. Processing data at this scale with conventional row-by-row R code is slow and memory hungry.

**Objective:** build a high-performance and scalable analytics workflow in R that

- prepares and joins ~20 million trip records with the taxi-zone lookup table using **data.table**,
- identifies travel patterns, high-demand periods, high-revenue routes, fare/distance characteristics and payment preferences,
- compares **functional programming** ( `apply()` , `lapply()` , `purrr::map()` ) with **vectorized** and **data.table** implementations,
- compares **sequential** and **parallel** execution ( `foreach` + `doParallel` ) on month-wise partitions,

- benchmarks every approach with `microbenchmark` / `system.time()` and memory measurements, and
- critically evaluates the approaches for execution time, memory, scalability and readability.

# 2. Dataset

| Item | Description |
|---|---|
| Source | NYC TLC Trip Record Data – https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page |
| Trip files | Yellow Taxi Trip Records, **January – June 2024** (6 monthly Parquet files, ≈ 20 million trips) |
| Lookup table | Taxi Zone Lookup Table ( `taxi_zone_lookup.csv` , 265 zones) + Taxi Zone shapefile (for the map) |

Main attributes used:

| Column | Meaning |
|---|---|
| `tpep_pickup_datetime` , `tpep_dropoff_datetime` | Pickup / drop-off date and time |
| `PULocationID` , `DOLocationID` | TLC taxi-zone IDs of pickup / drop-off |
| `passenger_count` | Number of passengers |
| `trip_distance` | Trip distance in miles |
| `payment_type` | 1 = Credit card, 2 = Cash, 3 = No charge, 4 = Dispute, 0 = Flex fare / unknown |
| `fare_amount` , `tip_amount` , `tolls_amount` , `total_amount` | Monetary amounts in USD |

Six months were chosen because they give a sufficiently large dataset (≈ 3 million rows per month) so that the performance differences between conventional, optimized, sequential and parallel implementations are clearly visible, while still fitting in the memory of a normal laptop. Change `MONTHS` in the next cell to use fewer months on a machine with less than 8 GB RAM.

## 2.1 Environment setup

Install (if needed) and load the packages. `TZ = "UTC"` is set so that the timestamps, which the TLC stores as New York local time without a time zone, are kept exactly as recorded.

The notebook runs on a local machine and on **Google Colab with an R runtime** (Runtime → Change runtime type → R). On Colab it automatically:

- analyses **three months instead of six**, because a Colab session has about 12 GB of RAM while six months need roughly 11 GB at peak — edit `MONTHS` below to

change this;

- installs the system libraries that the `sf` mapping package needs;
- uses **at least two worker cores**, since a Colab session has only two.

The optional map section (`sf` + `leaflet`) is skipped automatically if those packages cannot be installed, so the rest of the notebook always runs.

In [1]:

```r
ON_COLAB <- dir.exists("/content")                          # Google Colab R runtime
CRAN     <- "https://cloud.r-project.org"
have     <- function(p) requireNamespace(p, quietly = TRUE)
need     <- function(pkgs) { p <- pkgs[!vapply(pkgs, have, logical(1))]
                             if (length(p)) install.packages(p, repos = CRAN, quiet = TRUE) }

# 1. core packages used by every section
need(c("data.table", "ggplot2", "foreach", "doParallel", "microbenchmark",
       "purrr", "lubridate", "scales", "hexbin"))

# 2. a Parquet reader: arrow preferred, nanoparquet as a lightweight fallback
need("arrow")
if (!have("arrow")) need("nanoparquet")
HAVE_ARROW <- have("arrow")
stopifnot("need either arrow or nanoparquet to read Parquet files" =
HAVE_ARROW || have("nanoparquet"))

# 3. optional mapping packages (advanced task in Task 6); sf needs system libraries on Colab
if (ON_COLAB && !have("sf")) system("apt-get install -y -qq libgdal-dev libgeos-dev libproj-dev libudunits2-dev")
try(need(c("sf", "leaflet", "htmlwidgets")), silent = TRUE)
HAVE_MAPS <- all(vapply(c("sf", "leaflet", "htmlwidgets"), have, logical(1)))

Sys.setenv(TZ = "UTC")
Sys.setenv(ARROW_DEFAULT_MEMORY_POOL = "system")   # let arrow return memory to the OS after import
suppressPackageStartupMessages({
  library(data.table)
  if (HAVE_ARROW) library(arrow)
  library(ggplot2)
  library(foreach)
  library(doParallel)
  library(microbenchmark)
  library(purrr)
  library(lubridate)
  library(scales)
})

set.seed(2024)
options(repr.plot.width = 12, repr.plot.height = 6, scipen = 999, width = 140)

MONTHS   <- if (ON_COLAB) sprintf("2024-%02d", 1:3) else sprintf("2024-%02d", 1:6)
DATA_DIR <- if (ON_COLAB) "/content/data" else "data"
OUT_DIR  <- if (ON_COLAB) "/content/outputs" else "outputs"
```

```
FIG_DIR  <- file.path(OUT_DIR, "figures")
for (d in c(DATA_DIR, FIG_DIR)) dir.create(d, showWarnings = FALSE,
recursive = TRUE)

# keep one core free for the OS, but always use at least two workers so
that the
# sequential-vs-parallel comparison is meaningful on a 2-core machine such
as Colab
N_CORES <- max(2L, parallel::detectCores() - 1L)

cat("R version          :", R.version.string, "\n")
cat("data.table version :", as.character(packageVersion("data.table")),
"\n")
cat("Running on Colab   :", ON_COLAB, "| months analysed:",
length(MONTHS), "\n")
cat("Parquet reader     :", if (HAVE_ARROW) "arrow" else "nanoparquet",
    "| mapping packages available:", HAVE_MAPS, "\n")
cat("Logical cores      :", parallel::detectCores(), "| cores used for
parallel work:", N_CORES, "\n")
cat("data.table threads :", getDTthreads(), "\n")
```

```
R version          : R version 4.6.1 (2026-06-24)
data.table version : 1.18.6.1
Running on Colab   : FALSE | months analysed: 6
Parquet reader     : arrow | mapping packages available: TRUE
Logical cores      : 4 | cores used for parallel work: 3
data.table threads : 2
```

## 2.2 Helper functions

- `peak_mem_mb()` – peak extra memory (MB) used by R while evaluating an expression, measured with `gc()`.
- `save_plot()` – saves a ggplot to `outputs/figures/` and returns it so it is also shown in the notebook.

In [2]:

```
peak_mem_mb <- function(expr) {
  invisible(gc(reset = TRUE))
  before <- sum(gc()[, 2])                  # MB in use now
  force(expr)
  peak <- sum(gc()[, 6])                    # max MB used since reset
  round(peak - before, 1)
}

save_plot <- function(p, name, width = 11, height = 6) {
  ggsave(file.path(FIG_DIR, paste0(name, ".png")), p, width = width,
height = height, dpi = 120, bg = "white")
  p
}

theme_set(theme_minimal(base_size = 13) +
          theme(plot.title = element_text(face = "bold"), legend.position
= "bottom"))
```

## 2.3 Download the data

Files are downloaded once from the official TLC CloudFront location and re-used afterwards (the `data/` folder is not committed to Git).

In [3]:
```
options(timeout = 3600)
base_url   <- "https://d37ci6vzurychx.cloudfront.net"
trip_files <- file.path(DATA_DIR, sprintf("yellow_tripdata_%s.parquet",
MONTHS))

for (i in seq_along(MONTHS)) {
  if (!file.exists(trip_files[i])) {
    download.file(sprintf("%s/trip-data/yellow_tripdata_%s.parquet",
base_url, MONTHS[i]),
                  trip_files[i], mode = "wb")
  }
}
zone_file <- file.path(DATA_DIR, "taxi_zone_lookup.csv")
if (!file.exists(zone_file)) download.file(paste0(base_url,
"/misc/taxi_zone_lookup.csv"), zone_file, mode = "wb")
shape_zip <- file.path(DATA_DIR, "taxi_zones.zip")
if (!file.exists(shape_zip)) download.file(paste0(base_url,
"/misc/taxi_zones.zip"), shape_zip, mode = "wb")

data.table(file = basename(c(trip_files, zone_file, shape_zip)),
           size_MB = round(file.size(c(trip_files, zone_file, shape_zip))
/ 2^20, 1))
```

A data.table: 8 × 2

| file | size_MB |
|---|---|
| <chr> | <dbl> |
| yellow_tripdata_2024-01.parquet | 47.6 |
| yellow_tripdata_2024-02.parquet | 48.0 |
| yellow_tripdata_2024-03.parquet | 57.3 |
| yellow_tripdata_2024-04.parquet | 56.4 |
| yellow_tripdata_2024-05.parquet | 59.7 |
| yellow_tripdata_2024-06.parquet | 57.1 |
| taxi_zone_lookup.csv | 0.0 |
| taxi_zones.zip | 1.0 |

---

# 3. Task 1 – Data Acquisition and Preparation

## 3.1 Import the trip records

Each monthly Parquet file is read with `arrow` (or `nanoparquet` when arrow is unavailable) and the monthly tables are stacked with `data.table::rbindlist()`. Iterating over **files** with `lapply()` is a good use of functional programming: the loop body runs only a handful of times, so the per-call overhead is irrelevant — unlike looping over 20 million rows, which Task 3 measures.

In [4]:
```
read_trip_file <- function(f) {
  if (HAVE_ARROW) as.data.frame(arrow::read_parquet(f)) else
```

```
as.data.frame(nanoparquet::read_parquet(f))
}
t_import <- system.time({
  monthly_tables <- lapply(trip_files, read_trip_file)   # functional
iteration over the monthly files
  trips <- rbindlist(monthly_tables, fill = TRUE)        # data.table:
stack them into one table
  rm(monthly_tables); invisible(gc())
})
cat(sprintf("Imported %s rows x %d columns in %.1f s\n",
            format(nrow(trips), big.mark = ","), ncol(trips),
t_import[["elapsed"]]))
raw_size <- object.size(trips)
cat("Memory used by raw table:", format(raw_size, units = "auto"), "\n")
```

```
Imported 20,332,093 rows x 19 columns in 31.2 s
Memory used by raw table: 2.4 Gb
```

## 3.2 Inspect structure, dimensions, data types and summary statistics

In [5]:
```
dim(trips)
str(trips)
```

20332093 · 19

```
Classes ‘data.table’ and 'data.frame':  20332093 obs. of  19 variables:
 $ VendorID             : int  2 1 1 1 1 1 2 1 2 2 ...
 $ tpep_pickup_datetime : POSIXct, format: "2024-01-01 00:57:55" "2024-01-01 00:03:00" "2024-01-01
00:17:06" "2024-01-01 00:36:38" ...
 $ tpep_dropoff_datetime: POSIXct, format: "2024-01-01 01:17:43" "2024-01-01 00:09:36" "2024-01-01
00:35:01" "2024-01-01 00:44:56" ...
 $ passenger_count      : int  1 1 1 1 1 1 2 0 1 1 ...
 $ trip_distance        : num  1.72 1.8 4.7 1.4 0.8 ...
 $ RatecodeID           : int  1 1 1 1 1 1 1 1 1 1 ...
 $ store_and_fwd_flag   : chr  "N" "N" "N" "N" ...
 $ PULocationID         : int  186 140 236 79 211 148 138 246 161 113 ...
 $ DOLocationID         : int  79 236 79 211 148 141 181 231 261 113 ...
 $ payment_type         : int  2 1 1 1 1 1 1 2 2 2 ...
 $ fare_amount          : num  17.7 10 23.3 10 7.9 29.6 45.7 25.4 31 3 ...
 $ extra                : num  1 3.5 3.5 3.5 3.5 3.5 6 3.5 1 1 ...
 $ mta_tax              : num  0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 ...
 $ tip_amount           : num  0 3.75 3 2 3.2 6.9 10 0 0 0 ...
 $ tolls_amount         : num  0 0 0 0 0 0 0 0 0 0 ...
 $ improvement_surcharge: num  1 1 1 1 1 1 1 1 1 1 ...
 $ total_amount         : num  22.7 18.8 31.3 17 16.1 ...
 $ congestion_surcharge : num  2.5 2.5 2.5 2.5 2.5 2.5 0 2.5 2.5 2.5 ...
 $ Airport_fee          : num  0 0 0 0 0 0 1.75 0 0 0 ...
 - attr(*, ".internal.selfref")=<pointer: 0x560f95113480>
```

In [6]:
```
num_cols <- names(trips)[vapply(trips, is.numeric, logical(1))]
stats <- rbindlist(lapply(num_cols, function(cn) {
  x <- trips[[cn]]
  data.table(column = cn, min = min(x, na.rm = TRUE), mean = mean(x, na.rm
= TRUE),
             median = median(x, na.rm = TRUE), max = max(x, na.rm = TRUE),
missing = sum(is.na(x)))
}))
stats[, (2:5) := lapply(.SD, round, 2), .SDcols = 2:5]
stats
cat("Pickup time range:", format(range(trips$tpep_pickup_datetime)), "\n")
```

A data.table: 16 × 6

| column | min | mean | median | max | missing |
|---|---|---|---|---|---|
| <chr> | <dbl> | <dbl> | <dbl> | <dbl> | <int> |
| VendorID | 1.00 | 1.76 | 2.00 | 6.00 | 0 |
| passenger_count | 0.00 | 1.33 | 1.00 | 9.00 | 1975985 |
| trip_distance | 0.00 | 4.70 | 1.75 | 312722.30 | 0 |
| RatecodeID | 1.00 | 2.23 | 1.00 | 99.00 | 1975985 |
| PULocationID | 1.00 | 164.80 | 162.00 | 265.00 | 0 |
| DOLocationID | 1.00 | 163.90 | 162.00 | 265.00 | 0 |
| payment_type | 0.00 | 1.10 | 1.00 | 5.00 | 0 |
| fare_amount | -1285.40 | 18.89 | 13.50 | 334076.32 | 0 |
| extra | -7.50 | 1.41 | 1.00 | 65.99 | 0 |
| mta_tax | -0.50 | 0.48 | 0.50 | 35.84 | 0 |
| tip_amount | -300.00 | 3.27 | 2.60 | 999.99 | 0 |
| tolls_amount | -117.26 | 0.55 | 0.00 | 1702.88 | 0 |
| improvement_surcharge | -1.00 | 0.97 | 1.00 | 1.00 | 0 |
| total_amount | -1315.97 | 27.44 | 20.64 | 334145.30 | 0 |
| congestion_surcharge | -2.50 | 2.25 | 2.50 | 2.52 | 1975985 |
| Airport_fee | -1.75 | 0.14 | 0.00 | 1.75 | 1975985 |

```
Pickup time range: 2002-12-31 16:46:07 2026-06-26 23:53:12
```

## 3.3 Missing, duplicate, inconsistent and invalid records

First we count missing values per column.

In [7]:
```
na_counts <- trips[, lapply(.SD, function(x) sum(is.na(x)))]
na_counts <- melt(na_counts, measure.vars = names(na_counts),
variable.name = "column", value.name = "missing")
na_counts[, pct := round(100 * missing / nrow(trips), 2)]
na_counts[missing > 0]
# Rows with missing passenger_count also miss RatecodeID /
store_and_fwd_flag and have payment_type 0
trips[is.na(passenger_count), .N, by = payment_type]
```

A data.table: 5 × 3

| column | missing | pct |
|---|---|---|
| <fct> | <int> | <dbl> |
| passenger_count | 1975985 | 9.72 |
| RatecodeID | 1975985 | 9.72 |
| store_and_fwd_flag | 1975985 | 9.72 |
| congestion_surcharge | 1975985 | 9.72 |
| Airport_fee | 1975985 | 9.72 |

A data.table: 1 × 2

| payment_type | N |
|---|---|
| <int> | <int> |
| 0 | 1975985 |

**Cleaning rules** (each step is logged):

| Problem | Rule | Action |
|---|---|---|
| Duplicates | identical vendor, timestamps, locations, distance and amount | remove |
| Out-of-period records | pickup outside 1 Jan – 30 Jun 2024 (wrong meter clocks) | remove |
| Missing `passenger_count` / `RatecodeID` | ≈ 10 % of rows (flex-fare / app trips) | **impute** passenger count with the median (1), RatecodeID with 99 = unknown |
| Zero passengers | inconsistent with a paid trip | impute with median (1) |
| Non-positive or extreme amounts | fare ≤ 0 or > 500 USD, total ≤ 0, negative tip/tolls | remove (refunds, voids, typos) |
| Invalid distance | distance ≤ 0 or > 100 miles | remove |
| Invalid duration | drop-off not after pickup or trip longer than 3 hours | remove |
| Impossible speed | average speed > 80 mph | remove |
| Invalid zone IDs | outside 1–265 | remove |

In [8]:
```
# Memory-efficient cleaning: every rule only updates one logical "keep"
vector (80 MB for 20M rows)
# and the large table is subset ONCE at the end, instead of being copied
after every rule.
n_raw     <- nrow(trips)
keep      <- rep(TRUE, n_raw)
clean_log <- data.table(step = "Raw records", removed = 0L)
apply_rule <- function(step, ok) {
  ok[is.na(ok)] <- FALSE
  clean_log <<- rbind(clean_log, data.table(step = step, removed =
```

```
sum(keep & !ok)))
  keep <<- keep & ok
}

# trip duration in minutes (vectorized, added by reference)
trips[, trip_duration_min := as.numeric(tpep_dropoff_datetime -
tpep_pickup_datetime, units = "mins")]

period_start <- as.POSIXct("2024-01-01 00:00:00", tz = "UTC")
period_end   <- as.POSIXct("2024-07-01 00:00:00", tz = "UTC")

apply_rule("Remove duplicate trips", !duplicated(trips, by = c("VendorID",
"tpep_pickup_datetime", "tpep_dropoff_datetime",

"PULocationID", "DOLocationID", "trip_distance", "total_amount")))
apply_rule("Remove pickups outside Jan-Jun 2024", trips[,
tpep_pickup_datetime >= period_start & tpep_pickup_datetime < period_end])
apply_rule("Remove invalid fare / total / tip / tolls",
           trips[, fare_amount > 0 & fare_amount <= 500 & total_amount > 0
& tip_amount >= 0 & tolls_amount >= 0])
apply_rule("Remove invalid trip distance", trips[, trip_distance > 0 &
trip_distance <= 100])
apply_rule("Remove invalid trip duration", trips[, trip_duration_min > 0 &
trip_duration_min <= 180])
apply_rule("Remove speed > 80 mph", trips[, trip_distance /
(trip_duration_min / 60) <= 80])
apply_rule("Remove invalid zone IDs", trips[, PULocationID %between% c(1L,
265L) & DOLocationID %between% c(1L, 265L)])

trips <- trips[keep]          # the only copy of the large table
rm(keep); invisible(gc())

# impute missing / zero passenger count and missing rate code (update by
reference)
n_pass_fixed <- trips[is.na(passenger_count) | passenger_count == 0, .N]
trips[is.na(passenger_count) | passenger_count == 0, passenger_count :=
1L]
trips[is.na(RatecodeID), RatecodeID := 99L]
cat("Passenger count imputed for", format(n_pass_fixed, big.mark = ","),
"rows\n")

clean_log[, rows_left := n_raw - cumsum(removed)]
clean_log[, pct_of_raw := round(100 * rows_left / n_raw, 2)]
clean_log
```

```
Passenger count imputed for 2,001,287 rows
```

A data.table: 8 × 4

| step | removed | rows_left | pct_of_raw |
|---|---|---|---|
| <chr> | <int> | <int> | <dbl> |
| Raw records | 0 | 20332093 | 100.00 |
| Remove duplicate trips | 1 | 20332092 | 100.00 |
| Remove pickups outside Jan-Jun 2024 | 39 | 20332053 | 100.00 |
| Remove invalid fare / total / tip / tolls | 324323 | 20007730 | 98.40 |
| Remove invalid trip distance | 338584 | 19669146 | 96.74 |
| Remove invalid trip duration | 13090 | 19656056 | 96.68 |
| Remove speed > 80 mph | 5840 | 19650216 | 96.65 |
| Remove invalid zone IDs | 0 | 19650216 | 96.65 |

## 3.4 Convert attributes into appropriate data types

Integer codes become labelled **factors** (stored as integers + labels – memory efficient and readable).

In [9]:
```
payment_labels <- c("0" = "Flex/Unknown", "1" = "Credit card", "2" =
"Cash",
                    "3" = "No charge", "4" = "Dispute", "5" = "Unknown",
"6" = "Voided")
rate_labels <- c("1" = "Standard", "2" = "JFK", "3" = "Newark", "4" =
"Nassau/Westchester",
                 "5" = "Negotiated", "6" = "Group ride", "99" = "Unknown")

trips[, payment_type := factor(payment_labels[as.character(payment_type)],
                               levels = c("Credit card", "Cash",
"Flex/Unknown", "Dispute", "No charge", "Unknown", "Voided"))]
trips[, RatecodeID := factor(rate_labels[as.character(RatecodeID)], levels
= unique(rate_labels))]
trips[, passenger_count := as.integer(passenger_count)]
trips[, `:=`(PULocationID = as.integer(PULocationID), DOLocationID =
as.integer(DOLocationID))]
sapply(trips[, .(payment_type, RatecodeID, passenger_count, PULocationID,
trip_distance)], class)
```

**payment_type:** 'factor' **RatecodeID:** 'factor' **passenger_count:** 'integer' **PULocationID:** 'integer' **trip_distance:** 'numeric'

## 3.5 Extract temporal attributes

Hour, day of month, day of week (Monday first), month, date and a weekend flag. data.table's `hour()`, `mday()`, `wday()` and `month()` are fast vectorized C implementations.

In [10]:
```
wday_labels <- c("Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun")
t_time <- system.time({
  trips[, `:=`(
    pickup_date  = as.IDate(tpep_pickup_datetime),
```

```
    pickup_hour  = hour(tpep_pickup_datetime),
    pickup_day   = mday(tpep_pickup_datetime),
    pickup_wday  = factor((wday(tpep_pickup_datetime) + 5L) %% 7L + 1L,
levels = 1:7, labels = wday_labels),
    month        = factor(month(tpep_pickup_datetime), levels = 1:6,
labels = month.abb[1:6])
  )]
  trips[, is_weekend := pickup_wday %in% c("Sat", "Sun")]
  trips[, time_period := cut(pickup_hour, breaks = c(-1, 5, 9, 15, 19,
23),
                             labels = c("Early morning (0-5)", "Morning
rush (6-9)", "Midday (10-15)",
                                        "Evening rush (16-19)", "Night (20-
23)"))]
})
cat("Temporal features created in", t_time[["elapsed"]], "s\n")
head(trips[, .(tpep_pickup_datetime, pickup_date, pickup_hour, pickup_day,
pickup_wday, month, is_weekend, time_period)])
```

```
Temporal features created in 55.084 s
```

A data.table: 6 × 8

| tpep_pickup_datetime | pickup_date | pickup_hour | pickup_day | pickup_wday | month | is_w |
|---|---|---|---|---|---|---|
| <dttm> | <IDate> | <int> | <int> | <fct> | <fct> | |
| 2024-01-01 00:57:55 | 2024-01-01 | 0 | 1 | Mon | Jan | |
| 2024-01-01 00:03:00 | 2024-01-01 | 0 | 1 | Mon | Jan | |
| 2024-01-01 00:17:06 | 2024-01-01 | 0 | 1 | Mon | Jan | |
| 2024-01-01 00:36:38 | 2024-01-01 | 0 | 1 | Mon | Jan | |
| 2024-01-01 00:46:51 | 2024-01-01 | 0 | 1 | Mon | Jan | |
| 2024-01-01 00:54:08 | 2024-01-01 | 0 | 1 | Mon | Jan | |

## 3.6 Remove unnecessary attributes to reduce memory

Surcharges and technical flags are not needed for the analysis. Columns are dropped **by reference** ( `:= NULL` ), which does not copy the table.

In [11]:
```
size_before <- object.size(trips)
drop_cols <- intersect(c("VendorID", "store_and_fwd_flag", "extra",
"mta_tax", "improvement_surcharge",
                         "congestion_surcharge", "Airport_fee",
"airport_fee"), names(trips))
trips[, (drop_cols) := NULL]
size_after <- object.size(trips)
```

```
invisible(gc())
data.table(stage = c("Raw import", "After cleaning + new features", "After
dropping columns"),
           columns = c(19, ncol(trips) + length(drop_cols), ncol(trips)),
           size = c(format(raw_size, units = "MB"), format(size_before,
units = "MB"), format(size_after, units = "MB")))
```

A data.table: 3 × 3

| stage | columns | size |
|---|---|---|
| <chr> | <dbl> | <chr> |
| Raw import | 19 | 2482 Mb |
| After cleaning + new features | 27 | 3073.4 Mb |
| After dropping columns | 20 | 2098.9 Mb |

## 3.7 Import the Taxi Zone Lookup Table

In [12]:
```
zones <- fread(zone_file)
setkey(zones, LocationID)
cat("Zones:", nrow(zones), "\n")
zones[, .N, by = Borough][order(-N)]
head(zones)
```

```
Zones: 265
```

A data.table: 8 × 2

| Borough | N |
|---|---|
| <chr> | <int> |
| Queens | 69 |
| Manhattan | 69 |
| Brooklyn | 61 |
| Bronx | 43 |
| Staten Island | 20 |
| EWR | 1 |
| Unknown | 1 |
| N/A | 1 |

A data.table: 6 × 4

| LocationID | Borough | Zone | service_zone |
|---|---|---|---|
| <int> | <chr> | <chr> | <chr> |
| 1 | EWR | Newark Airport | EWR |
| 2 | Queens | Jamaica Bay | Boro Zone |
| 3 | Bronx | Allerton/Pelham Gardens | Boro Zone |
| 4 | Manhattan | Alphabet City | Yellow Zone |
| 5 | Staten Island | Arden Heights | Boro Zone |
| 6 | Staten Island | Arrochar/Fort Wadsworth | Boro Zone |

## 3.8 Join trips with zone information

An **update join** ( `X[Y, on = , := ]` ) adds pickup and drop-off borough/zone to the 20 million rows by reference – no copy of the large table is created (a `merge()` would copy it twice).

In [13]:
```
t_join <- system.time({
  trips[zones, on = .(PULocationID = LocationID), `:=`(pu_borough =
i.Borough, pu_zone = i.Zone)]
  trips[zones, on = .(DOLocationID = LocationID), `:=`(do_borough =
i.Borough, do_zone = i.Zone)]
  # repeated strings -> factors to save memory
  for (col in c("pu_borough", "pu_zone", "do_borough", "do_zone"))
set(trips, j = col, value = factor(trips[[col]]))
})
cat("Two update joins on", format(nrow(trips), big.mark = ","), "rows
took", t_join[["elapsed"]], "s\n")
setkey(trips, tpep_pickup_datetime)
trips[, .(trips = .N), by = pu_borough][order(-trips)]
```

```
Two update joins on 19,650,216 rows took 20.451 s
```

A data.table: 8 × 2

| pu_borough | trips |
|---|---|
| <fct> | <int> |
| Manhattan | 17574156 |
| Queens | 1723780 |
| Brooklyn | 234405 |
| Unknown | 60625 |
| Bronx | 52261 |
| N/A | 4009 |
| Staten Island | 520 |
| EWR | 460 |

## 3.9 Final prepared dataset

In [14]:
```
str(trips)
cat(sprintf("\nFinal dataset: %s trips (%.1f %% of raw records kept), %s
in memory\n",
            format(nrow(trips), big.mark = ","), 100 * nrow(trips) /
n_raw,
            format(object.size(trips), units = "auto")))
```

```
Classes ‘data.table’ and 'data.frame':  19650216 obs. of  24 variables:
 $ tpep_pickup_datetime : POSIXct, format: "2024-01-01 00:00:00" "2024-01-01 00:00:02" "2024-01-01
00:00:03" "2024-01-01 00:00:04" ...
 $ tpep_dropoff_datetime: POSIXct, format: "2024-01-01 00:02:49" "2024-01-01 00:04:36" "2024-01-01
00:03:16" "2024-01-01 00:22:32" ...
 $ passenger_count      : int  1 2 1 1 2 1 1 1 1 1 ...
 $ trip_distance        : num  0.3 1.57 0.5 7.37 18.85 ...
 $ RatecodeID           : Factor w/ 7 levels "Standard","JFK",..: 1 1 1 1 2 4 1 1 1 1 ...
 $ PULocationID         : int  140 161 136 125 132 234 229 113 161 162 ...
 $ DOLocationID         : int  140 236 18 157 41 265 181 234 75 236 ...
 $ payment_type         : Factor w/ 7 levels "Credit card",..: 1 1 2 2 1 2 1 1 1 1 ...
 $ fare_amount          : num  4.4 8.6 5.1 33.8 70 81.4 38 16.3 12.1 14.9 ...
 $ tip_amount           : num  1.85 2.72 0 0 15.69 ...
 $ tolls_amount         : num  0 0 0 0 6.94 ...
 $ total_amount         : num  11.2 16.3 7.6 38.8 95.9 ...
 $ trip_duration_min    : num  2.82 4.57 3.22 22.47 26.65 ...
 $ pickup_date          : IDate, format: "2024-01-01" "2024-01-01" "2024-01-01" "2024-01-01" ...
 $ pickup_hour          : int  0 0 0 0 0 0 0 0 0 0 ...
 $ pickup_day           : int  1 1 1 1 1 1 1 1 1 1 ...
 $ pickup_wday          : Factor w/ 7 levels "Mon","Tue","Wed",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ month                : Factor w/ 6 levels "Jan","Feb","Mar",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ is_weekend           : logi  FALSE FALSE FALSE FALSE FALSE FALSE ...
 $ time_period          : Factor w/ 5 levels "Early morning (0-5)",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ pu_borough           : Factor w/ 8 levels "Bronx","Brooklyn",..: 4 4 1 4 6 4 4 4 4 4 ...
 $ pu_zone              : Factor w/ 260 levels "Allerton/Pelham Gardens",..: 134 155 130 118 122 231 226
106 155 156 ...
 $ do_borough           : Factor w/ 8 levels "Bronx","Brooklyn",..: 4 4 1 6 4 5 2 4 4 4 ...
 $ do_zone              : Factor w/ 260 levels "Allerton/Pelham Gardens",..: 135 233 16 152 39 177 179
231 72 233 ...
 - attr(*, ".internal.selfref")=<pointer: 0x560f95113480>
 - attr(*, "sorted")= chr "tpep_pickup_datetime"
Final dataset: 19,650,216 trips (96.6 % of raw records kept), 2.3 Gb in memory
```

**Interpretation:** after removing duplicates, out-of-period, invalid and implausible records about 97 % of the raw records remain, so the cleaning removes noise without biasing the dataset. Converting codes to factors and dropping unused columns reduces the memory footprint, and every expensive step (feature creation, imputation, joins) was done **by reference** with data.table, which avoids copying a multi-gigabyte table.

---

# 4. Task 2 – Transportation Data Analytics

All aggregations use data.table's `DT[i, j, by]` syntax.

## 4.1 Number of taxi trips by hour of day

In [15]:
```
hourly <- trips[, .(trips = .N, avg_fare = mean(fare_amount), avg_total =
mean(total_amount)), by = pickup_hour][order(pickup_hour)]
hourly[, share_pct := round(100 * trips / sum(trips), 2)]
hourly
cat("Peak hour:", hourly[which.max(trips), pickup_hour], ":00 with",
format(max(hourly$trips), big.mark = ","), "trips\n")
cat("Quietest hour:", hourly[which.min(trips), pickup_hour], ":00 with",
format(min(hourly$trips), big.mark = ","), "trips\n")
```

A data.table: 24 × 5

| pickup_hour | trips | avg_fare | avg_total | share_pct |
|---|---|---|---|---|
| <int> | <int> | <dbl> | <dbl> | <dbl> |
| 0 | 561076 | 19.75567 | 28.25432 | 2.86 |
| 1 | 363176 | 17.72769 | 25.63441 | 1.85 |
| 2 | 235892 | 16.58758 | 24.23721 | 1.20 |
| 3 | 155337 | 17.78036 | 25.60346 | 0.79 |
| 4 | 111660 | 23.06086 | 31.63272 | 0.57 |
| 5 | 120201 | 26.86042 | 36.74176 | 0.61 |
| 6 | 272913 | 22.64153 | 30.85086 | 1.39 |
| 7 | 546596 | 19.39535 | 27.13617 | 2.78 |
| 8 | 757665 | 18.58747 | 26.21087 | 3.86 |
| 9 | 829123 | 18.52194 | 26.41832 | 4.22 |
| 10 | 882863 | 18.79553 | 26.90593 | 4.49 |
| 11 | 956840 | 18.86350 | 26.90027 | 4.87 |
| 12 | 1047428 | 19.17166 | 27.27790 | 5.33 |
| 13 | 1086352 | 19.96273 | 28.29526 | 5.53 |
| 14 | 1166101 | 20.67340 | 29.22118 | 5.93 |
| 15 | 1196675 | 20.64434 | 29.16584 | 6.09 |
| 16 | 1213118 | 20.61005 | 30.96470 | 6.17 |
| 17 | 1338016 | 19.24176 | 29.19176 | 6.81 |
| 18 | 1408831 | 18.07551 | 27.78585 | 7.17 |
| 19 | 1236569 | 18.22104 | 27.97929 | 6.29 |
| 20 | 1116640 | 18.34523 | 27.28326 | 5.68 |
| 21 | 1149071 | 18.47963 | 27.30065 | 5.85 |
| 22 | 1072132 | 19.25034 | 28.00551 | 5.46 |
| 23 | 825941 | 20.38919 | 29.23294 | 4.20 |

```
Peak hour: 18 :00 with 1,408,831 trips
Quietest hour: 4 :00 with 111,660 trips
```

## 4.2 Taxi demand across days of the week

Because a six-month period does not contain the same number of Mondays, Tuesdays … , demand is also normalised as **average trips per day**.

In [16]:
```
dow <- trips[, .(trips = .N, days = uniqueN(pickup_date), avg_fare =
round(mean(fare_amount), 2),
                 avg_distance = round(mean(trip_distance), 2)), by =
pickup_wday][order(pickup_wday)]
dow[, avg_trips_per_day := round(trips / days)]
```

```
dow
dow_hour <- trips[, .N, by = .(is_weekend, pickup_hour)]
dow_hour[, days := ifelse(is_weekend, trips[is_weekend == TRUE,
uniqueN(pickup_date)], trips[is_weekend == FALSE, uniqueN(pickup_date)])]
dow_hour[, trips_per_day := N / days]
dcast(dow_hour, pickup_hour ~ ifelse(is_weekend, "weekend", "weekday"),
value.var = "trips_per_day")[, lapply(.SD, round)]
```

A data.table: 7 × 6

| pickup_wday | trips | days | avg_fare | avg_distance | avg_trips_per_day |
|---|---|---|---|---|---|
| <fct> | <int> | <int> | <dbl> | <dbl> | <dbl> |
| Mon | 2351136 | 26 | 20.07 | 3.73 | 90428 |
| Tue | 2755577 | 26 | 19.20 | 3.27 | 105984 |
| Wed | 2963594 | 26 | 19.19 | 3.20 | 113984 |
| Thu | 3159268 | 26 | 19.53 | 3.27 | 121510 |
| Fri | 2938155 | 26 | 19.12 | 3.30 | 113006 |
| Sat | 2993326 | 26 | 18.23 | 3.12 | 115128 |
| Sun | 2489160 | 26 | 20.07 | 3.77 | 95737 |

A data.table: 24 × 3

| pickup_hour | weekday | weekend |
|---|---|---|
| <dbl> | <dbl> | <dbl> |
| 0 | 1949 | 5918 |
| 1 | 923 | 4678 |
| 2 | 508 | 3266 |
| 3 | 339 | 2139 |
| 4 | 372 | 1218 |
| 5 | 709 | 539 |
| 6 | 1760 | 848 |
| 7 | 3686 | 1296 |
| 8 | 5009 | 2048 |
| 9 | 5094 | 3210 |
| 10 | 5074 | 4293 |
| 11 | 5312 | 5122 |
| 12 | 5714 | 5858 |
| 13 | 5889 | 6169 |
| 14 | 6480 | 6226 |
| 15 | 6709 | 6241 |
| 16 | 6739 | 6481 |
| 17 | 7648 | 6611 |
| 18 | 8191 | 6617 |
| 19 | 7074 | 6094 |
| 20 | 6478 | 5279 |
| 21 | 6750 | 5223 |
| 22 | 6154 | 5233 |
| 23 | 4423 | 4826 |

## 4.3 Monthly variation in taxi usage

In [17]:
```
monthly <- trips[, .(trips = .N, days = uniqueN(pickup_date), revenue_musd
= round(sum(total_amount) / 1e6, 2),
                     avg_fare = round(mean(fare_amount), 2), avg_distance
= round(mean(trip_distance), 2),
                     avg_duration_min = round(mean(trip_duration_min),
1)), by = month][order(month)]
monthly[, trips_per_day := round(trips / days)]
monthly[, change_vs_prev_pct := round(100 * (trips_per_day /
shift(trips_per_day) - 1), 1)]
monthly
```

A data.table: 6 × 9

| month | trips | days | revenue_musd | avg_fare | avg_distance | avg_duration_min | trips_ |
|---|---|---|---|---|---|---|---|
| <fct> | <int> | <int> | <dbl> | <dbl> | <dbl> | <dbl> | |
| Jan | 2866574 | 31 | 78.33 | 18.48 | 3.29 | 15.0 | |
| Feb | 2898586 | 29 | 78.90 | 18.40 | 3.22 | 15.4 | |
| Mar | 3436527 | 31 | 95.70 | 19.14 | 3.36 | 16.1 | |
| Apr | 3410774 | 30 | 96.10 | 19.44 | 3.39 | 16.4 | |
| May | 3612954 | 31 | 104.74 | 20.13 | 3.43 | 17.5 | |
| Jun | 3424801 | 30 | 98.20 | 19.93 | 3.42 | 17.0 | |

## 4.4 Average fare and total revenue for different time periods

In [18]:
```
period_stats <- trips[, .(trips = .N, avg_fare = round(mean(fare_amount),
2), avg_total = round(mean(total_amount), 2),
                          avg_distance = round(mean(trip_distance), 2),
revenue_musd = round(sum(total_amount) / 1e6, 2)),
                       by = time_period][order(time_period)]
period_stats[, revenue_share_pct := round(100 * revenue_musd /
sum(revenue_musd), 1)]
period_stats
trips[, .(trips = .N, avg_fare = round(mean(fare_amount), 2), avg_total =
round(mean(total_amount), 2),
          revenue_musd = round(sum(total_amount) / 1e6, 2)), by = .
(day_type = ifelse(is_weekend, "Weekend", "Weekday"))]
```

A data.table: 5 × 7

| time_period | trips | avg_fare | avg_total | avg_distance | revenue_musd | revenue_share_ |
|---|---|---|---|---|---|---|
| <fct> | <int> | <dbl> | <dbl> | <dbl> | <dbl> | <dl |
| Early morning (0-5) | 1547342 | 19.39 | 27.66 | 3.79 | 42.81 | |
| Morning rush (6-9) | 2406297 | 19.21 | 27.02 | 3.45 | 65.02 | 1 |
| Midday (10-15) | 6336259 | 19.76 | 28.06 | 3.29 | 177.78 | 3 |
| Evening rush (16-19) | 5196534 | 19.00 | 28.94 | 3.15 | 150.37 | 2 |
| Night (20-23) | 4163784 | 19.02 | 27.86 | 3.51 | 116.01 | 2 |

A data.table: 2 × 5

| day_type | trips | avg_fare | avg_total | revenue_musd |
|---|---|---|---|---|
| <chr> | <int> | <dbl> | <dbl> | <dbl> |
| Weekday | 14167730 | 19.40 | 28.46 | 403.17 |
| Weekend | 5482486 | 19.07 | 27.14 | 148.81 |

## 4.5 Most frequently travelled pickup – drop-off routes

In [19]:
```
routes <- trips[, .(trips = .N, avg_fare = mean(fare_amount), avg_distance
= mean(trip_distance),
                    revenue = sum(total_amount)), by = .(pu_zone,
do_zone)]
top_routes <- routes[order(-trips)][1:15]
top_routes[, `:=`(avg_fare = round(avg_fare, 2), avg_distance =
round(avg_distance, 2), revenue_kusd = round(revenue / 1e3))][, revenue :=
NULL]
top_routes
cat("Distinct routes observed:", format(nrow(routes), big.mark = ","),
"\n")
cat("Share of trips starting and ending in the same zone:",
    round(100 * trips[PULocationID == DOLocationID, .N] / nrow(trips), 2),
"%\n")
```

A data.table: 15 × 6

| pu_zone | do_zone | trips | avg_fare | avg_distance | revenue_kusd |
|---|---|---|---|---|---|
| <fct> | <fct> | <int> | <dbl> | <dbl> | <dbl> |
| Upper East Side South | Upper East Side North | 137973 | 8.87 | 1.06 | 2169 |
| Upper East Side North | Upper East Side South | 120122 | 9.37 | 1.04 | 1937 |
| Upper East Side North | Upper East Side North | 92602 | 6.87 | 0.64 | 1228 |
| Upper East Side South | Upper East Side South | 89270 | 7.47 | 0.65 | 1244 |
| Midtown Center | Upper East Side South | 65481 | 10.12 | 1.07 | 1132 |
| Upper East Side South | Midtown Center | 57371 | 10.36 | 1.06 | 983 |
| Midtown Center | Upper East Side North | 55540 | 14.14 | 1.94 | 1225 |
| Upper West Side South | Lincoln Square East | 53721 | 7.92 | 0.89 | 783 |
| Lincoln Square East | Upper West Side South | 53093 | 8.11 | 1.00 | 796 |
| Upper East Side South | Midtown East | 51072 | 9.56 | 0.99 | 833 |
| Upper West Side South | Upper West Side North | 50263 | 7.26 | 0.84 | 697 |
| Lenox Hill West | Upper East Side North | 46049 | 9.09 | 1.09 | 726 |
| Upper East Side North | Midtown Center | 45924 | 14.88 | 1.93 | 1013 |
| JFK Airport | Times Sq/Theatre District | 43128 | 70.40 | 17.99 | 4067 |
| Upper West Side North | Upper West Side South | 42653 | 7.23 | 0.80 | 584 |

```
Distinct routes observed: 43,932
Share of trips starting and ending in the same zone: 4.2 %
```

## 4.6 Highest-revenue routes (total amount collected)

In [20]:

```
top_rev_routes <- routes[order(-revenue)][1:15]
top_rev_routes[, `:=`(revenue_musd = round(revenue / 1e6, 2), avg_fare =
round(avg_fare, 2),
                      avg_distance = round(avg_distance, 1),
revenue_per_trip = round(revenue / trips, 2))][, revenue := NULL]
top_rev_routes
airport_zones <- c("JFK Airport", "LaGuardia Airport", "Newark Airport")
trips[, .(trips = .N, revenue_musd = round(sum(total_amount) / 1e6, 1)),
      by = .(airport_trip = pu_zone %in% airport_zones | do_zone %in%
airport_zones)][
```

```
      , `:=`(trip_share_pct = round(100 * trips / sum(trips), 1),
revenue_share_pct = round(100 * revenue_musd / sum(revenue_musd), 1))][]
```

A data.table: 15 × 7

| pu_zone <fct> | do_zone <fct> | trips <int> | avg_fare <dbl> | avg_distance <dbl> | revenue_musd <dbl> | revenue_per_trip <dbl> |
|---|---|---|---|---|---|---|
| JFK Airport | Outside of NYC | 35131 | 106.83 | 21.0 | 4.47 | 127.25 |
| JFK Airport | Times Sq/Theatre District | 43128 | 70.40 | 18.0 | 4.07 | 94.31 |
| LaGuardia Airport | Times Sq/Theatre District | 39937 | 49.53 | 10.3 | 3.12 | 78.20 |
| JFK Airport | Midtown South | 22726 | 70.29 | 17.3 | 2.17 | 95.59 |
| Upper East Side South | Upper East Side North | 137973 | 8.87 | 1.1 | 2.17 | 15.72 |
| JFK Airport | Clinton East | 22425 | 70.44 | 18.2 | 2.10 | 93.71 |
| Times Sq/Theatre District | LaGuardia Airport | 26450 | 50.12 | 11.3 | 2.01 | 75.99 |
| Upper East Side North | Upper East Side South | 120122 | 9.37 | 1.0 | 1.94 | 16.13 |
| LaGuardia Airport | Midtown Center | 24343 | 46.71 | 10.0 | 1.84 | 75.46 |
| JFK Airport | Midtown Center | 18362 | 70.33 | 17.7 | 1.76 | 95.59 |
| Times Sq/Theatre District | JFK Airport | 18701 | 70.58 | 17.6 | 1.73 | 92.30 |
| JFK Airport | Midtown North | 17836 | 70.45 | 18.3 | 1.68 | 94.08 |
| JFK Airport | Murray Hill | 16971 | 70.23 | 16.9 | 1.62 | 95.41 |
| LaGuardia Airport | Outside of NYC | 10549 | 111.95 | 22.3 | 1.50 | 141.87 |
| JFK Airport | East Chelsea | 15002 | 70.37 | 18.1 | 1.43 | 95.61 |

A data.table: 2 × 5

| airport_trip <lgl> | trips <int> | revenue_musd <dbl> | trip_share_pct <dbl> | revenue_share_pct <dbl> |
|---|---|---|---|---|
| FALSE | 17735539 | 402.1 | 90.3 | 72.8 |
| TRUE | 1914677 | 149.9 | 9.7 | 27.2 |

## 4.7 Relationship between trip distance and fare amount

In [21]:
```
cat("Pearson correlation (distance, fare)  :", round(trips[,
cor(trip_distance, fare_amount)], 3), "\n")
samp_cor <- trips[sample(.N, 1e6), .(trip_distance, fare_amount)]
cat("Spearman correlation (1M-row sample)   :", round(samp_cor[,
cor(trip_distance, fare_amount, method = "spearman")], 3), "\n")

fit <- lm(fare_amount ~ trip_distance, data = trips[RatecodeID ==
"Standard", .(fare_amount, trip_distance)][sample(.N, 1e6)])
cat(sprintf("Linear model (standard rate, 1M-row sample): fare = %.2f +
%.2f x miles, R^2 = %.3f\n",
            coef(fit)[1], coef(fit)[2], summary(fit)$r.squared))

dist_bands <- trips[, .(trips = .N, avg_fare = round(mean(fare_amount),
2), median_fare = median(fare_amount),
                        fare_per_mile = round(mean(fare_amount /
trip_distance), 2),
                        avg_duration_min = round(mean(trip_duration_min),
1)),
                    by = .(distance_band = cut(trip_distance, c(0, 1, 2,
3, 5, 10, 20, 100),
                                               labels = c("0-1", "1-2",
"2-3", "3-5", "5-10", "10-20", "20+")))][order(distance_band)]
dist_bands
```

```
Pearson correlation (distance, fare)  : 0.936
Spearman correlation (1M-row sample)   : 0.911
Linear model (standard rate, 1M-row sample): fare = 6.15 + 3.88 x miles, R^2 = 0.918
```

A data.table: 7 × 6

| distance_band | trips | avg_fare | median_fare | fare_per_mile | avg_duration_min |
|---|---|---|---|---|---|
| <fct> | <int> | <dbl> | <dbl> | <dbl> | <dbl> |
| 0-1 | 4541685 | 8.12 | 7.20 | 28.67 | 6.4 |
| 1-2 | 6435665 | 12.03 | 11.40 | 8.30 | 11.3 |
| 2-3 | 3143726 | 17.00 | 16.30 | 6.97 | 16.5 |
| 3-5 | 2261809 | 22.95 | 21.94 | 6.07 | 21.6 |
| 5-10 | 1730511 | 34.60 | 33.80 | 4.84 | 27.6 |
| 10-20 | 1350971 | 61.45 | 66.81 | 4.18 | 44.2 |
| 20+ | 185849 | 90.72 | 70.00 | 3.71 | 55.2 |

## 4.8 Payment methods across boroughs

In [22]:
```
pay_overall <- trips[, .(trips = .N, avg_total = round(mean(total_amount),
2)), by = payment_type][order(-trips)]
pay_overall[, share_pct := round(100 * trips / sum(trips), 2)]
pay_overall
pay_borough <- trips[!pu_borough %in% c("Unknown", "N/A"), .N, by = .
(pu_borough, payment_type)]
pay_borough[, share_pct := round(100 * N / sum(N), 1), by = pu_borough]
dcast(pay_borough[payment_type %in% c("Credit card", "Cash",
"Flex/Unknown")], pu_borough ~ payment_type, value.var = "share_pct", fill
= 0)
```

A data.table: 5 × 4

| payment_type | trips | avg_total | share_pct |
|---|---|---|---|
| <fct> | <int> | <dbl> | <dbl> |
| Credit card | 14973411 | 29.10 | 76.20 |
| Cash | 2652440 | 24.47 | 13.50 |
| Flex/Unknown | 1789762 | 25.30 | 9.11 |
| Dispute | 164196 | 26.73 | 0.84 |
| No charge | 70407 | 23.86 | 0.36 |

A data.table: 6 × 4

| pu_borough | Credit card | Cash | Flex/Unknown |
|---|---|---|---|
| <fct> | <dbl> | <dbl> | <dbl> |
| Bronx | 57.0 | 5.3 | 37.0 |
| Brooklyn | 50.9 | 5.9 | 42.4 |
| EWR | 86.7 | 8.9 | 0.4 |
| Manhattan | 76.5 | 13.2 | 9.2 |
| Queens | 76.8 | 17.9 | 3.2 |
| Staten Island | 11.5 | 34.8 | 49.6 |

## 4.9 Additional patterns

In [23]:
```
# (a) tipping behaviour by payment type (cash tips are not recorded by the
meter)
trips[payment_type %in% c("Credit card", "Cash"),
      .(avg_tip = round(mean(tip_amount), 2), tip_pct_of_fare = round(100
* mean(tip_amount / fare_amount), 1),
        zero_tip_pct = round(100 * mean(tip_amount == 0), 1)), by =
payment_type]

# (b) average speed by hour - congestion pattern
speed_hour <- trips[trip_duration_min >= 1, .(avg_speed_mph =
round(mean(trip_distance / (trip_duration_min / 60)), 1)), by =
pickup_hour][order(pickup_hour)]
cat("Slowest hour:", speed_hour[which.min(avg_speed_mph), pickup_hour],
":00 (", min(speed_hour$avg_speed_mph), "mph ) | fastest hour:",
    speed_hour[which.max(avg_speed_mph), pickup_hour], ":00 (",
max(speed_hour$avg_speed_mph), "mph )\n")
```

```r
# (c) passenger count distribution
trips[, .(trips = .N), by = .(passengers = pmin(passenger_count, 5L))]
[order(passengers)][, share_pct := round(100 * trips / sum(trips), 1)][]

# (d) days with the highest and lowest demand
daily <- trips[, .(trips = .N, revenue = sum(total_amount)), by =
pickup_date][order(pickup_date)]
rbind(daily[order(-trips)][1:3][, type := "busiest"], daily[order(trips)]
[1:3][, type := "quietest"])
```

A data.table: 2 × 4

| payment_type | avg_tip | tip_pct_of_fare | zero_tip_pct |
|---|---|---|---|
| <fct> | <dbl> | <dbl> | <dbl> |
| Credit card | 4.28 | 25.3 | 5.2 |
| Cash | 0.00 | 0.0 | 100.0 |

```
Slowest hour: 15 :00 ( 9.3 mph ) | fastest hour: 5 :00 ( 19.6 mph )
```

A data.table: 5 × 3

| passengers | trips | share_pct |
|---|---|---|
| <int> | <int> | <dbl> |
| 1 | 15825911 | 80.5 |
| 2 | 2581756 | 13.1 |
| 3 | 592529 | 3.0 |
| 4 | 341493 | 1.7 |
| 5 | 308527 | 1.6 |

A data.table: 6 × 4

| pickup_date | trips | revenue | type |
|---|---|---|---|
| <IDate> | <int> | <dbl> | <chr> |
| 2024-05-16 | 139761 | 4165499 | busiest |
| 2024-04-11 | 138371 | 3866001 | busiest |
| 2024-06-06 | 137626 | 4081325 | busiest |
| 2024-01-07 | 65125 | 1866363 | quietest |
| 2024-02-13 | 67097 | 1701203 | quietest |
| 2024-05-27 | 70119 | 2098582 | quietest |

**Interpretation of Task 2**

- **Hourly demand** follows a clear commuter/leisure cycle: demand is lowest around 4–5 AM, builds through the morning and peaks in the late afternoon / early evening (≈ 17–18 h). Average fares are highest in the early morning because those trips are longer (airport and outer-borough runs).
- **Day of week:** Thursday has the highest average trips per day, followed by Saturday, Wednesday and Friday; Monday and Sunday are the quietest (several

Monday public holidays fall in this period). The weekday/weekend hourly profile shows weekend demand shifted to late night (0–3 h).

- **Monthly:** demand rises from January to spring (March–May are the busiest), reflecting winter seasonality and tourism; June stays high.
- **Revenue by period:** the midday (≈ 32 %) and evening-rush (≈ 27 %) periods generate most of the revenue; the amount per trip is similar in all periods (≈ 27–29 USD), but early-morning trips are the longest.
- **Routes:** the most frequent routes are short trips inside Manhattan's Upper East Side; the **highest-revenue routes all involve JFK / LaGuardia airports** – airport trips are a small share of trips but a much larger share of revenue.
- **Distance vs fare:** a strong positive, almost linear relationship (correlation ≈ 0.94; under the standard rate fare ≈ 6 USD + ≈ 3.9 USD per mile, $R^2$ ≈ 0.92), but the **fare per mile falls** sharply with distance because of the fixed base charge and time-based charges on short congested trips.
- **Payment:** credit cards dominate (about three quarters of trips), cash is ≈ 13 %. Cash is used more in Queens than in Manhattan, while Bronx and Brooklyn pickups have a very high "flex/unknown" share (no payment type recorded). Recorded tips are almost exclusively on card payments, because cash tips are not metered.
- **Other patterns:** taxis are slowest in the afternoon (congestion) and fastest early in the morning; most trips carry a single passenger; holidays and storms produce the quietest days.

---

# 5. Task 3 – Functional vs Vectorized vs data.table Programming

The same analytical operations are implemented in several styles and compared with `microbenchmark` (execution time) and `peak_mem_mb()` (peak extra memory). Row-wise functional code is far too slow for 20 million rows, so a **random sample of 200,000 trips** is used for the row-level experiment and **2 million trips** for the grouping experiment.

In [24]:
```
samp_dt <- trips[sample(.N, 2e5), .(fare_amount, trip_distance,
tip_amount, total_amount, pu_borough, pickup_hour)]
samp_df <- as.data.frame(samp_dt)          # plain data.frame for the
base-R versions
big_dt  <- trips[sample(.N, 2e6), .(fare_amount, trip_distance,
tip_amount, total_amount, pu_borough, pickup_hour)]
big_df  <- as.data.frame(big_dt)
cat("Sample sizes:", nrow(samp_df), "and", nrow(big_df), "rows\n")
bench_results <- list()   # collects every benchmark for Task 5
add_bench <- function(mb, comparison, n_rows, mem = NULL) {
  s <- as.data.table(summary(mb, unit = "ms"))[, .(approach =
as.character(expr), median_ms = round(median, 3), mean_ms = round(mean,
3))]
  s[, `:=`(comparison = comparison, rows = n_rows)]
  if (!is.null(mem)) s[, peak_mem_MB := mem[approach]]
  bench_results[[comparison]] <<- s
```

```
  s[, relative := round(median_ms / min(median_ms), 1)][order(median_ms)]
}
```

```
Sample sizes: 200000 and 2000000 rows
```

## 5.1 Experiment A – row-level calculation (fare per mile)

Compute `fare_amount / trip_distance` for every trip.

In [25]:

```
fpm_apply  <- function(df) apply(df[, c("fare_amount", "trip_distance")],
1, function(r) r[1] / r[2])
fpm_lapply <- function(df) unlist(lapply(seq_len(nrow(df)), function(i)
df$fare_amount[i] / df$trip_distance[i]))
fpm_map    <- function(df) purrr::map2_dbl(df$fare_amount,
df$trip_distance, function(f, d) f / d)
fpm_loop   <- function(df) { out <- numeric(nrow(df)); for (i in
seq_len(nrow(df))) out[i] <- df$fare_amount[i] / df$trip_distance[i]; out
}
fpm_vector <- function(df) df$fare_amount / df$trip_distance
fpm_dt     <- function(dt) dt[, fare_amount / trip_distance]

# all implementations give the same answer
ref <- fpm_vector(samp_df)
stopifnot(all.equal(unname(fpm_apply(samp_df)), ref),
all.equal(fpm_lapply(samp_df), ref),
          all.equal(fpm_map(samp_df), ref), all.equal(fpm_loop(samp_df),
ref), all.equal(fpm_dt(samp_dt), ref))

mem_A <- c(`apply()` = peak_mem_mb(fpm_apply(samp_df)), `lapply()` =
peak_mem_mb(fpm_lapply(samp_df)),
           `purrr::map2_dbl()` = peak_mem_mb(fpm_map(samp_df)), `for loop`
= peak_mem_mb(fpm_loop(samp_df)),
           `vectorized` = peak_mem_mb(fpm_vector(samp_df)), `data.table` =
peak_mem_mb(fpm_dt(samp_dt)))

mb_A <- microbenchmark(`apply()` = fpm_apply(samp_df), `lapply()` =
fpm_lapply(samp_df),
                       `purrr::map2_dbl()` = fpm_map(samp_df), `for loop`
= fpm_loop(samp_df),
                       `vectorized` = fpm_vector(samp_df), `data.table` =
fpm_dt(samp_dt), times = 10)
add_bench(mb_A, "A: row-level fare per mile", nrow(samp_df), mem_A)
```

A data.table: 6 × 7

| approach | median_ms | mean_ms | comparison | rows | peak_mem_MB | relative |
|---|---|---|---|---|---|---|
| <chr> | <dbl> | <dbl> | <chr> | <int> | <dbl> | <dbl> |
| vectorized | 0.465 | 0.461 | A: row-level fare per mile | 200000 | 2.2 | 1.0 |
| data.table | 1.525 | 1.706 | A: row-level fare per mile | 200000 | 2.5 | 3.3 |
| purrr::map2_dbl() | 406.028 | 387.948 | A: row-level fare per mile | 200000 | 167.8 | 873.2 |
| for loop | 481.637 | 498.786 | A: row-level fare per mile | 200000 | 182.4 | 1035.8 |
| lapply() | 898.496 | 896.448 | A: row-level fare per mile | 200000 | 184.9 | 1932.2 |
| apply() | 1572.686 | 1652.990 | A: row-level fare per mile | 200000 | 198.4 | 3382.1 |

## 5.2 Experiment B – grouped aggregation (mean total amount per borough and hour)

In [26]:

```
grp_lapply <- function(df) {
  g <- split(df$total_amount, list(df$pu_borough, df$pickup_hour), drop =
TRUE)
  unlist(lapply(g, mean))
}
grp_map    <- function(df) purrr::map_dbl(split(df$total_amount,
list(df$pu_borough, df$pickup_hour), drop = TRUE), mean)
grp_sapply <- function(df) {           # conventional: filter the data
once per group
  keys <- unique(df[, c("pu_borough", "pickup_hour")])
  sapply(seq_len(nrow(keys)), function(k)
mean(df$total_amount[df$pu_borough == keys$pu_borough[k] & df$pickup_hour
== keys$pickup_hour[k]]))
}
grp_tapply    <- function(df) tapply(df$total_amount, list(df$pu_borough,
df$pickup_hour), mean)
grp_aggregate <- function(df) aggregate(total_amount ~ pu_borough +
pickup_hour, data = df, FUN = mean)
grp_dt        <- function(dt) dt[, .(mean_total = mean(total_amount)), by
= .(pu_borough, pickup_hour)]

# check: same group means from functional and data.table versions
chk <- grp_dt(big_dt)[order(pu_borough, pickup_hour)]
stopifnot(all.equal(unname(grp_lapply(big_df)[paste(chk$pu_borough,
chk$pickup_hour, sep = ".")]), chk$mean_total))

mem_B <- c(`lapply(split())` = peak_mem_mb(grp_lapply(big_df)),
`purrr::map_dbl(split())` = peak_mem_mb(grp_map(big_df)),
           `sapply() filter per group` = peak_mem_mb(grp_sapply(big_df)),
```

```
`tapply()` = peak_mem_mb(grp_tapply(big_df)),
          `aggregate()` = peak_mem_mb(grp_aggregate(big_df)), `data.table
by` = peak_mem_mb(grp_dt(big_dt)))

mb_B <- microbenchmark(`lapply(split())` = grp_lapply(big_df),
`purrr::map_dbl(split())` = grp_map(big_df),
                       `sapply() filter per group` = grp_sapply(big_df),
`tapply()` = grp_tapply(big_df),
                       `aggregate()` = grp_aggregate(big_df), `data.table
by` = grp_dt(big_dt), times = 5)
add_bench(mb_B, "B: grouped mean by borough x hour", nrow(big_df), mem_B)
```

A data.table: 6 × 7

| approach | median_ms | mean_ms | comparison | rows | peak_mem_MB | rela |
|---|---|---|---|---|---|---|
| <chr> | <dbl> | <dbl> | <chr> | <int> | <dbl> | <c |
| data.table by | 48.519 | 50.144 | B: grouped mean by borough x hour | 2000000 | 57.6 | |
| tapply() | 154.821 | 293.529 | B: grouped mean by borough x hour | 2000000 | 77.6 | |
| lapply(split()) | 273.292 | 291.086 | B: grouped mean by borough x hour | 2000000 | 248.2 | |
| purrr::map_dbl(split()) | 402.979 | 498.468 | B: grouped mean by borough x hour | 2000000 | 246.3 | |
| aggregate() | 844.985 | 1123.304 | B: grouped mean by borough x hour | 2000000 | 601.2 | |
| sapply() filter per group | 13625.624 | 13793.609 | B: grouped mean by borough x hour | 2000000 | 3863.7 | 28 |

## 5.3 Experiment C – summary statistics over many columns

Here the functional approach iterates over **columns** (only a handful of iterations), not rows.

In [27]:
```
num_vars <- c("fare_amount", "trip_distance", "tip_amount",
"total_amount")
col_apply  <- function(df) apply(df[, num_vars], 2, mean)
col_lapply <- function(df) unlist(lapply(df[, num_vars], mean))
col_map    <- function(df) purrr::map_dbl(df[, num_vars], mean)
col_vector <- function(df) colMeans(df[, num_vars])
col_dt     <- function(dt) dt[, lapply(.SD, mean), .SDcols = num_vars]

stopifnot(all.equal(col_lapply(big_df), col_vector(big_df)),
```

```
all.equal(unlist(col_dt(big_dt)), col_vector(big_df)))
mem_C <- c(`apply(, 2)` = peak_mem_mb(col_apply(big_df)), `lapply()` =
peak_mem_mb(col_lapply(big_df)),
           `purrr::map_dbl()` = peak_mem_mb(col_map(big_df)), `colMeans()
vectorized` = peak_mem_mb(col_vector(big_df)),
           `data.table .SD` = peak_mem_mb(col_dt(big_dt)))
mb_C <- microbenchmark(`apply(, 2)` = col_apply(big_df), `lapply()` =
col_lapply(big_df), `purrr::map_dbl()` = col_map(big_df),
                       `colMeans() vectorized` = col_vector(big_df),
`data.table .SD` = col_dt(big_dt), times = 10)
add_bench(mb_C, "C: column means (4 columns)", nrow(big_df), mem_C)
```

A data.table: 5 × 7

| approach | median_ms | mean_ms | comparison | rows | peak_mem_MB | relative |
|---|---|---|---|---|---|---|
| <chr> | <dbl> | <dbl> | <chr> | <int> | <dbl> | <dbl> |
| purrr::map_dbl() | 20.733 | 21.384 | C: column means (4 columns) | 2000000 | 0.1 | 1.0 |
| lapply() | 20.946 | 20.741 | C: column means (4 columns) | 2000000 | 1.1 | 1.0 |
| data.table .SD | 24.494 | 23.701 | C: column means (4 columns) | 2000000 | 1.3 | 1.2 |
| colMeans() vectorized | 31.036 | 32.142 | C: column means (4 columns) | 2000000 | 61.1 | 1.5 |
| apply(, 2) | 92.768 | 94.497 | C: column means (4 columns) | 2000000 | 213.7 | 4.5 |

## 5.4 Comparison of code complexity and readability

In [28]:
```
data.table(
  approach      = c("apply()", "lapply() / sapply()", "purrr::map()", "for
loop", "Vectorized base R", "data.table"),
  lines_of_code = c(1, 1, 1, 3, 1, 1),
  readability   = c("Medium - matrix conversion hidden", "Medium", "High -
explicit, type-stable (map_dbl)",
                    "High for beginners, verbose", "Very high", "High once
DT[i, j, by] is learnt"),
  complexity    = c("Low", "Low", "Low", "Medium (pre-allocation needed)",
"Very low", "Low"),
  memory        = c("High - converts data.frame to matrix", "High - one R
call + list element per row",
                    "High per row, low per column", "Low", "Low - one
result vector", "Lowest - in-place, no copies"),
  large_data    = c("Unsuitable row-wise", "Unsuitable row-wise, fine over
columns/groups",
                    "Unsuitable row-wise, fine over columns/groups",
"Unsuitable", "Good", "Best (multi-threaded, by reference)")
)
```

A data.table: 6 × 6

| approach | lines_of_code | readability | complexity | memory | large_data |
|---|---|---|---|---|---|
| <chr> | <dbl> | <chr> | <chr> | <chr> | <chr> |
| apply() | 1 | Medium - matrix conversion hidden | Low | High - converts data.frame to matrix | Unsuitable row-wise |
| lapply() / sapply() | 1 | Medium | Low | High - one R call + list element per row | Unsuitable row-wise, fine over columns/groups |
| purrr::map() | 1 | High - explicit, type-stable (map_dbl) | Low | High per row, low per column | Unsuitable row-wise, fine over columns/groups |
| for loop | 3 | High for beginners, verbose | Medium (pre-allocation needed) | Low | Unsuitable |
| Vectorized base R | 1 | Very high | Very low | Low - one result vector | Good |
| data.table | 1 | High once DT[i, j, by] is learnt | Low | Lowest - in-place, no copies | Best (multi-threaded, by reference) |

**Why the performance differs**

- `apply()` first **coerces the data.frame to a matrix** (copying all values) and then calls an R function once per row – 200,000 interpreted function calls. `lapply()`, `purrr::map()` and `for` loops avoid the matrix copy but still pay the cost of one interpreted R call per element plus list allocation.
- **Vectorized** arithmetic (`df$fare / df$dist`) performs a single call into compiled C code that loops over the whole vector – typically **hundreds to thousands of times faster** than the row-wise versions.
- For **grouping**, `split()` + `lapply()` is reasonable (the loop is over groups, not rows) but `split()` copies the data into a list; the "filter per group" `sapply()` scans the full vector once per group; `aggregate()` has heavy formula/model-frame overhead. **data.table** groups with a radix-sort based algorithm in C, uses several threads (GForce optimisation for `mean()`), and allocates memory only for the result.
- When the functional loop runs over **a few columns** (Experiment C), the call overhead is negligible: `lapply()` / `map_dbl()` call the compiled `mean()` once per column and are as fast as data.table's `.SD` and even faster than `colMeans()`, because `colMeans()` and `apply(, 2)` must first convert the data.frame into a matrix (an extra copy). Functional style is the best choice there.

---

# 6. Task 4 – Parallel Computing with foreach + doParallel

## 6.1 Logical partitions

The dataset is divided into **month-wise partitions** (6 subsets of ≈ 3 million trips). Only the columns needed by the analysis are kept so that less data has to be sent to the worker processes.

In [29]:
```
rm(samp_dt, samp_df, big_dt, big_df, samp_cor, fit); invisible(gc())   # free Task 3 objects
parts <- split(trips[, .(month, pu_borough, fare_amount, total_amount, trip_distance, trip_duration_min, tip_amount)],
               by = "month", keep.by = TRUE, drop = TRUE)
data.table(month = names(parts), rows = sapply(parts, nrow), size_MB = round(sapply(parts, function(p) object.size(p)) / 2^20, 1))
```

A data.table: 6 × 3

| month | rows | size_MB |
|---|---|---|
| <chr> | <int> | <dbl> |
| Jan | 2866574 | 131.2 |
| Feb | 2898586 | 132.7 |
| Mar | 3436527 | 157.3 |
| Apr | 3410774 | 156.1 |
| May | 3612954 | 165.4 |
| Jun | 3424801 | 156.8 |

## 6.2 Computationally intensive operation – bootstrap analysis per month

For every month we estimate **95 % bootstrap confidence intervals** for the mean total amount, the median trip distance and the mean fare per mile ( `B` resamples of ≈ 3 million rows each), plus borough-level statistics. Resampling millions of rows many times is CPU-bound and each month is independent – a good candidate for parallelism.

In [30]:
```
analyse_partition <- function(d, B = 20) {
  data.table::setDTthreads(1)              # one thread per worker -> fair sequential vs parallel comparison
  set.seed(1000 + match(as.character(d$month[1]), month.abb))  # reproducible, identical in both modes
  n <- nrow(d)
  boot <- t(vapply(seq_len(B), function(b) {
    idx <- sample.int(n, n, replace = TRUE)
    c(mean_total    = mean(d$total_amount[idx]),
      median_dist   = median(d$trip_distance[idx]),
      fare_per_mile = mean(d$fare_amount[idx] / d$trip_distance[idx]))
  }, numeric(3)))
  ci <- apply(boot, 2, quantile, probs = c(0.025, 0.975))
```

```
  data.table::data.table(
    month = as.character(d$month[1]), trips = n,
    mean_total = mean(d$total_amount), mean_total_lo = ci[1,
"mean_total"], mean_total_hi = ci[2, "mean_total"],
    median_dist = median(d$trip_distance), median_dist_lo = ci[1,
"median_dist"], median_dist_hi = ci[2, "median_dist"],
    fare_per_mile = mean(d$fare_amount / d$trip_distance),
    manhattan_share = mean(d$pu_borough == "Manhattan"))
}
B <- 20
old_threads <- getDTthreads()
```

## 6.3 A. Sequential processing

In [31]:
```
t_seq <- system.time(res_seq <- rbindlist(lapply(parts, analyse_partition,
B = B)))
t_seq_foreach <- system.time(res_seq_fe <- foreach(d = parts, .combine =
rbind) %do% analyse_partition(d, B = B))
setDTthreads(old_threads)
cat(sprintf("Sequential lapply()      : %.2f s\nSequential foreach %%do%%
: %.2f s\n", t_seq[["elapsed"]], t_seq_foreach[["elapsed"]]))
res_seq[, lapply(.SD, function(x) if (is.numeric(x)) round(x, 3) else x)]
```

```
Sequential lapply()      : 54.77 s
Sequential foreach %do% : 52.95 s
```

A data.table: 6 × 10

| month | trips | mean_total | mean_total_lo | mean_total_hi | median_dist | median_dist_lo |
|---|---|---|---|---|---|---|
| <chr> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> |
| Jan | 2866574 | 27.326 | 27.291 | 27.341 | 1.70 | 1.70 |
| Feb | 2898586 | 27.221 | 27.202 | 27.257 | 1.72 | 1.71 |
| Mar | 3436527 | 27.848 | 27.819 | 27.861 | 1.80 | 1.80 |
| Apr | 3410774 | 28.175 | 28.143 | 28.191 | 1.81 | 1.81 |
| May | 3612954 | 28.989 | 28.972 | 29.009 | 1.81 | 1.81 |
| Jun | 3424801 | 28.674 | 28.651 | 28.693 | 1.82 | 1.81 |

## 6.4 B. Parallel processing with foreach + doParallel

A PSOCK cluster (works on Windows, macOS and Linux) is created with `N_CORES` workers; `foreach(... ) %dopar%` sends one monthly partition to each free worker and `rbind`s the results.

In [32]:
```
t_cluster <- system.time({
  cl <- makeCluster(N_CORES)
  registerDoParallel(cl)
  invisible(clusterEvalQ(cl, library(data.table)))
})
cat("Workers registered:", getDoParWorkers(), "| cluster start-up:",
t_cluster[["elapsed"]], "s\n")

t_par <- system.time(
  res_par <- foreach(d = parts, .combine = rbind, .packages =
"data.table") %dopar%
```

```
    analyse_partition(d, B = B)
)
cat(sprintf("Parallel foreach %%dopar%% with %d workers: %.2f s\n",
N_CORES, t_par[["elapsed"]]))
stopifnot(all.equal(res_seq, res_par, check.attributes = FALSE))   #
identical results
cat("Results identical to sequential run: TRUE\n")
```

```
Workers registered: 3 | cluster start-up: 0.33 s
Parallel foreach %dopar% with 3 workers: 25.90 s
Results identical to sequential run: TRUE
```

## 6.5 Speedup and scaling with the number of cores

$$\text{Speedup} = \frac{\text{Sequential execution time}}{\text{Parallel execution time}} \qquad \text{Efficiency} = \frac{\text{Speedup}}{\text{cores}}$$

In [33]:
```
stopCluster(cl)
core_grid <- unique(c(1L, 2L, seq(2L, N_CORES, by = 2L), N_CORES))
scaling <- rbindlist(lapply(core_grid, function(k) {
  cl_k <- makeCluster(k); registerDoParallel(cl_k)
  invisible(clusterEvalQ(cl_k, library(data.table)))
  el <- system.time(foreach(d = parts, .combine = rbind, .packages =
"data.table", .export = c("analyse_partition", "B")) %dopar%
                      analyse_partition(d, B = B))[["elapsed"]]
  stopCluster(cl_k)
  data.table(cores = k, parallel_s = el)
}))
scaling[, `:=`(sequential_s = t_seq[["elapsed"]], speedup =
round(t_seq[["elapsed"]] / parallel_s, 2))]
scaling[, efficiency_pct := round(100 * speedup / cores)]
scaling
speedup_main <- round(t_seq[["elapsed"]] / t_par[["elapsed"]], 2)
cat(sprintf("\nSpeedup with %d cores = %.2f s / %.2f s = %.2fx\n",
N_CORES, t_seq[["elapsed"]], t_par[["elapsed"]], speedup_main))
```

A data.table: 3 × 5

| cores | parallel_s | sequential_s | speedup | efficiency_pct |
|---|---|---|---|---|
| <int> | <dbl> | <dbl> | <dbl> | <dbl> |
| 1 | 58.088 | 54.767 | 0.94 | 94 |
| 2 | 30.901 | 54.767 | 1.77 | 88 |
| 3 | 21.915 | 54.767 | 2.50 | 83 |

```
Speedup with 3 cores = 54.77 s / 25.90 s = 2.11x
```

## 6.6 When parallelism does not help – a lightweight task

Computing a simple mean per month is so cheap that sending ≈ 600 MB of partitions to the workers costs far more than the computation itself.

In [34]:
```
light_task <- function(d) data.table::data.table(month =
as.character(d$month[1]), mean_fare = mean(d$fare_amount))
cl <- makeCluster(N_CORES); registerDoParallel(cl)
t_light_seq <- system.time(foreach(d = parts, .combine = rbind) %do%
light_task(d))[["elapsed"]]
t_light_par <- system.time(foreach(d = parts, .combine = rbind, .packages
```

```
= "data.table") %dopar% light_task(d))[["elapsed"]]
t_light_dt  <- system.time(trips[, .(mean_fare = mean(fare_amount)), by =
month])[["elapsed"]]
stopCluster(cl); registerDoSEQ()
light <- data.table(approach = c("Sequential foreach %do%",
sprintf("Parallel foreach %%dopar%% (%d cores)", N_CORES), "data.table by
= month (no partitions)"),
                    elapsed_s = c(t_light_seq, t_light_par, t_light_dt))
light[, speedup_vs_seq := round(t_light_seq / elapsed_s, 2)]
light
```

A data.table: 3 × 3

| approach | elapsed_s | speedup_vs_seq |
|---|---|---|
| <chr> | <dbl> | <dbl> |
| Sequential foreach %do% | 0.063 | 1.00 |
| Parallel foreach %dopar% (3 cores) | 5.270 | 0.01 |
| data.table by = month (no partitions) | 0.350 | 0.18 |

**Discussion**

- For the **CPU-intensive bootstrap**, the parallel version is clearly faster; the speedup grows with the number of cores but stays **below the ideal linear speedup** (efficiency < 100 %) because of (1) cluster start-up, (2) serialising each ≈ 100 MB partition to the worker processes, (3) only 6 partitions for N workers (load imbalance: with 3 workers the last "round" is uneven when 6 is not a multiple of the core count) and (4) memory-bandwidth contention.
- For the **lightweight task** parallel execution is *slower* than sequential: the work per partition takes milliseconds while copying the data to workers takes seconds. A single data.table grouping on the full table (which is itself multi-threaded) is faster than both.
- Therefore parallel processing pays off only when the computation per partition is large compared with the communication overhead.

---

# 7. Task 5 – Performance Benchmarking and Optimization

## 7.1 Base R vs data.table on the full dataset

Typical operations on all ≈ 20 million trips: filtering, grouped aggregation, sorting and joining. A base-R `data.frame` copy of the needed columns is used for the base-R versions.

In [35]:
```
rm(parts); invisible(gc())   # free Task 4 partitions
full_dt <- trips[, .(PULocationID, pickup_hour, month, fare_amount,
total_amount, trip_distance)]
full_df <- setDF(copy(full_dt))   # plain data.frame for the base-R
versions
```

```
zones_df <- as.data.frame(zones)
n_full <- nrow(full_df)

invisible(gc())
cat("data.table threads:", getDTthreads(), "\n")
mb_filter <- microbenchmark(
  `base R df[cond, ]`       = full_df[full_df$trip_distance > 10 &
full_df$fare_amount > 50, ],
  `base R subset()`         = subset(full_df, trip_distance > 10 &
fare_amount > 50),
  `data.table DT[cond]`     = full_dt[trip_distance > 10 & fare_amount >
50], times = 7)
add_bench(mb_filter, "D: filter long expensive trips", n_full)
```

```
data.table threads: 2
```

A data.table: 3 × 6

| approach | median_ms | mean_ms | comparison | rows | relative |
|---|---|---|---|---|---|
| <chr> | <dbl> | <dbl> | <chr> | <int> | <dbl> |
| data.table DT[cond] | 195.868 | 204.617 | D: filter long expensive trips | 19650216 | 1.0 |
| base R df[cond, ] | 524.372 | 551.222 | D: filter long expensive trips | 19650216 | 2.7 |
| base R subset() | 575.556 | 583.304 | D: filter long expensive trips | 19650216 | 2.9 |

In [36]:
```
invisible(gc())
mb_group <- microbenchmark(
  `base R aggregate()`      = aggregate(total_amount ~ month +
pickup_hour, data = full_df, FUN = sum),
  `base R tapply()`         = tapply(full_df$total_amount,
list(full_df$month, full_df$pickup_hour), sum),
  `data.table by`           = full_dt[, .(revenue = sum(total_amount)), by
= .(month, pickup_hour)], times = 7)
add_bench(mb_group, "E: revenue by month x hour", n_full)
```

A data.table: 3 × 6

| approach | median_ms | mean_ms | comparison | rows | relative |
|---|---|---|---|---|---|
| <chr> | <dbl> | <dbl> | <chr> | <int> | <dbl> |
| data.table by | 386.483 | 398.284 | E: revenue by month x hour | 19650216 | 1 |
| base R tapply() | 1154.110 | 1796.898 | E: revenue by month x hour | 19650216 | 3 |
| base R aggregate() | 15075.872 | 15323.157 | E: revenue by month x hour | 19650216 | 39 |

In [37]:
```
invisible(gc())
mb_sort <- microbenchmark(
  `base R order()`          = full_df[order(-full_df$total_amount), ],
  `data.table setorder()`   = setorder(copy(full_dt), -total_amount),
  `data.table DT[order()]`  = full_dt[order(-total_amount)], times = 5)
add_bench(mb_sort, "F: sort by total amount", n_full)
```

A data.table: 3 × 6

| approach | median_ms | mean_ms | comparison | rows | relative |
|---|---|---|---|---|---|
| <chr> | <dbl> | <dbl> | <chr> | <int> | <dbl> |
| data.table DT[order()] | 1944.107 | 2422.971 | F: sort by total amount | 19650216 | 1.0 |
| data.table setorder() | 3392.911 | 3291.744 | F: sort by total amount | 19650216 | 1.7 |
| base R order() | 7048.573 | 7751.149 | F: sort by total amount | 19650216 | 3.6 |

In [38]:
```
join_n  <- 2e6
join_df <- full_df[seq_len(join_n), ]
join_dt <- full_dt[seq_len(join_n)]
mem_join <- c(`base R merge()` = peak_mem_mb(merge(join_df, zones_df, by.x
= "PULocationID", by.y = "LocationID", all.x = TRUE)),
              `base R match()` = peak_mem_mb({j <- join_df; j$Borough <-
zones_df$Borough[match(j$PULocationID, zones_df$LocationID)]; j}),
              `data.table join` = peak_mem_mb(zones[join_dt, on = .
(LocationID = PULocationID)]),
              `data.table update join :=` = peak_mem_mb(join_dt[zones, on
= .(PULocationID = LocationID), Borough := i.Borough]))
mb_join <- microbenchmark(
  `base R merge()`          = merge(join_df, zones_df, by.x =
"PULocationID", by.y = "LocationID", all.x = TRUE),
  `base R match()`          = {j <- join_df; j$Borough <-
zones_df$Borough[match(j$PULocationID, zones_df$LocationID)]; j},
  `data.table join`         = zones[join_dt, on = .(LocationID =
PULocationID)],
  `data.table update join :=` = join_dt[zones, on = .(PULocationID =
LocationID), Borough := i.Borough], times = 3)
add_bench(mb_join, "G: join 2M trips with zones", join_n, mem_join)
```

A data.table: 4 × 7

| approach | median_ms | mean_ms | comparison | rows | peak_mem_MB | relative |
|---|---|---|---|---|---|---|
| <chr> | <dbl> | <dbl> | <chr> | <dbl> | <dbl> | <dbl> |
| base R match() | 31.553 | 29.929 | G: join 2M trips with zones | 2000000 | 30.6 | 1.0 |
| data.table join | 164.774 | 163.616 | G: join 2M trips with zones | 2000000 | 152.9 | 5.2 |
| data.table update join := | 238.510 | 229.070 | G: join 2M trips with zones | 2000000 | 76.8 | 7.6 |
| base R merge() | 4940.220 | 4810.649 | G: join 2M trips with zones | 2000000 | 675.1 | 156.6 |

## 7.2 Scalability – execution time vs number of rows

The grouped aggregation (mean fare by pickup zone) is run on increasing subsets.

In [39]:
```
sizes <- c(1e5, 5e5, 1e6, 5e6, 1e7, n_full)
scal <- rbindlist(lapply(sizes, function(n) {
  df <- full_df[seq_len(n), c("PULocationID", "fare_amount")]
  dt <- full_dt[seq_len(n), .(PULocationID, fare_amount)]
  t <- summary(microbenchmark(
    `lapply(split())` = lapply(split(df$fare_amount, df$PULocationID),
mean),
    `tapply()`        = tapply(df$fare_amount, df$PULocationID, mean),
    `aggregate()`     = aggregate(fare_amount ~ PULocationID, data = df,
FUN = mean),
    `data.table by`   = dt[, mean(fare_amount), by = PULocationID], times
= 3), unit = "ms")
  data.table(rows = n, approach = as.character(t$expr), median_ms =
t$median)
}))
dcast(scal, rows ~ approach, value.var = "median_ms")[, lapply(.SD, round,
1)]
```

A data.table: 6 × 5

| rows | aggregate() | data.table by | lapply(split()) | tapply() |
|---|---|---|---|---|
| <dbl> | <dbl> | <dbl> | <dbl> | <dbl> |
| 100000 | 42.8 | 5.1 | 4.0 | 8.4 |
| 500000 | 124.0 | 13.1 | 15.8 | 22.0 |
| 1000000 | 279.5 | 23.7 | 29.5 | 53.8 |
| 5000000 | 1587.2 | 105.5 | 193.6 | 269.1 |
| 10000000 | 6246.4 | 737.1 | 314.1 | 691.8 |
| 19650216 | 12845.9 | 488.7 | 808.6 | 984.8 |

## 7.3 Consolidated execution-time comparison table

In [40]:
```
par_tbl <- data.table(approach = c("Sequential lapply()", "Sequential
foreach %do%", sprintf("Parallel foreach %%dopar%% (%d cores)", N_CORES)),
                      median_ms = round(1000 * c(t_seq[["elapsed"]],
t_seq_foreach[["elapsed"]], t_par[["elapsed"]]), 1),
                      comparison = "H: bootstrap per month (sequential vs
parallel)", rows = nrow(trips))
bench_all <- rbindlist(c(bench_results, list(par_tbl)), fill = TRUE)
bench_all[, relative_to_best := round(median_ms / min(median_ms), 1), by =
comparison]
bench_all <- bench_all[order(comparison, median_ms), .(comparison, rows,
approach, median_ms, relative_to_best, peak_mem_MB)]
fwrite(bench_all, file.path(OUT_DIR, "benchmark_results.csv"))
fwrite(scaling, file.path(OUT_DIR, "parallel_scaling.csv"))
rm(full_df, full_dt, join_df, join_dt); invisible(gc())
bench_all
```

A data.table: 33 × 6

| comparison | rows | approach | median_ms | relative_to_best | peak_mem_MB |
|---|---|---|---|---|---|
| <chr> | <dbl> | <chr> | <dbl> | <dbl> | <dbl> |
| A: row-level fare per mile | 200000 | vectorized | 0.465 | 1.0 | 2.2 |
| A: row-level fare per mile | 200000 | data.table | 1.525 | 3.3 | 2.5 |
| A: row-level fare per mile | 200000 | purrr::map2_dbl() | 406.028 | 873.2 | 167.8 |
| A: row-level fare per mile | 200000 | for loop | 481.637 | 1035.8 | 182.4 |
| A: row-level fare per mile | 200000 | lapply() | 898.496 | 1932.2 | 184.9 |
| A: row-level fare per mile | 200000 | apply() | 1572.686 | 3382.1 | 198.4 |
| B: grouped mean by borough x hour | 2000000 | data.table by | 48.519 | 1.0 | 57.6 |
| B: grouped mean by borough x hour | 2000000 | tapply() | 154.821 | 3.2 | 77.6 |
| B: grouped mean by borough x hour | 2000000 | lapply(split()) | 273.292 | 5.6 | 248.2 |
| B: grouped mean by borough x hour | 2000000 | purrr::map_dbl(split()) | 402.979 | 8.3 | 246.3 |
| B: grouped mean by borough x hour | 2000000 | aggregate() | 844.985 | 17.4 | 601.2 |
| B: grouped mean by borough x hour | 2000000 | sapply() filter per group | 13625.624 | 280.8 | 3863.7 |
| C: column means (4 columns) | 2000000 | purrr::map_dbl() | 20.733 | 1.0 | 0.1 |
| C: column means (4 columns) | 2000000 | lapply() | 20.946 | 1.0 | 1.1 |

| comparison | rows | approach | median_ms | relative_to_best | peak_mem_MB |
|---|---|---|---|---|---|
| <chr> | <dbl> | <chr> | <dbl> | <dbl> | <dbl> |
| C: column means (4 columns) | 2000000 | data.table .SD | 24.494 | 1.2 | 1.3 |
| C: column means (4 columns) | 2000000 | colMeans() vectorized | 31.036 | 1.5 | 61.1 |
| C: column means (4 columns) | 2000000 | apply(, 2) | 92.768 | 4.5 | 213.7 |
| D: filter long expensive trips | 19650216 | data.table DT[cond] | 195.868 | 1.0 | NA |
| D: filter long expensive trips | 19650216 | base R df[cond, ] | 524.372 | 2.7 | NA |
| D: filter long expensive trips | 19650216 | base R subset() | 575.556 | 2.9 | NA |
| E: revenue by month x hour | 19650216 | data.table by | 386.483 | 1.0 | NA |
| E: revenue by month x hour | 19650216 | base R tapply() | 1154.110 | 3.0 | NA |
| E: revenue by month x hour | 19650216 | base R aggregate() | 15075.872 | 39.0 | NA |
| F: sort by total amount | 19650216 | data.table DT[order()] | 1944.107 | 1.0 | NA |
| F: sort by total amount | 19650216 | data.table setorder() | 3392.911 | 1.7 | NA |
| F: sort by total amount | 19650216 | base R order() | 7048.573 | 3.6 | NA |
| G: join 2M trips with zones | 2000000 | base R match() | 31.553 | 1.0 | 30.6 |
| G: join 2M trips with zones | 2000000 | data.table join | 164.774 | 5.2 | 152.9 |
| G: join 2M trips with zones | 2000000 | data.table update join := | 238.510 | 7.6 | 76.8 |

| comparison | rows | approach | median_ms | relative_to_best | peak_mem_MB |
|---|---|---|---|---|---|
| <chr> | <dbl> | <chr> | <dbl> | <dbl> | <dbl> |
| G: join 2M trips with zones | 2000000 | base R merge() | 4940.220 | 156.6 | 675.1 |
| H: bootstrap per month (sequential vs parallel) | 19650216 | Parallel foreach %dopar% (3 cores) | 25896.000 | 1.0 | NA |
| H: bootstrap per month (sequential vs parallel) | 19650216 | Sequential foreach %do% | 52949.000 | 2.0 | NA |
| H: bootstrap per month (sequential vs parallel) | 19650216 | Sequential lapply() | 54767.000 | 2.1 | NA |

In [41]:
```r
# head-to-head summary for the four required comparisons
best_of <- function(cmp, pattern) bench_all[comparison == cmp &
grepl(pattern, approach), min(median_ms)]
summary_tbl <- data.table(
  comparison = c("Base R vs data.table (grouping, 20M rows)", "Base R vs
data.table (join, 2M rows)",
                 "Functional vs vectorized (row-level)", "Functional vs
data.table (grouping)", "Sequential vs parallel (bootstrap)"),
  slower_approach_ms = c(best_of("E: revenue by month x hour",
"aggregate"), best_of("G: join 2M trips with zones", "merge"),
                         best_of("A: row-level fare per mile", "lapply"),
best_of("B: grouped mean by borough x hour", "lapply"),
                         1000 * t_seq[["elapsed"]]),
  faster_approach_ms = c(best_of("E: revenue by month x hour",
"data.table"), best_of("G: join 2M trips with zones", "data.table"),
                         best_of("A: row-level fare per mile",
"vectorized"), best_of("B: grouped mean by borough x hour", "data.table"),
                         1000 * t_par[["elapsed"]]))
summary_tbl[, speedup := round(slower_approach_ms / faster_approach_ms,
1)]
summary_tbl[, (2:3) := lapply(.SD, round, 1), .SDcols = 2:3]
summary_tbl
```

A data.table: 5 × 4

| comparison | slower_approach_ms | faster_approach_ms | speedup |
|---|---|---|---|
| <chr> | <dbl> | <dbl> | <dbl> |
| Base R vs data.table (grouping, 20M rows) | 15075.9 | 386.5 | 39.0 |
| Base R vs data.table (join, 2M rows) | 4940.2 | 164.8 | 30.0 |
| Functional vs vectorized (row-level) | 898.5 | 0.5 | 1932.2 |
| Functional vs data.table (grouping) | 273.3 | 48.5 | 5.6 |
| Sequential vs parallel (bootstrap) | 54767.0 | 25896.0 | 2.1 |

**Analysis**

- **Execution time:** data.table is the fastest approach for filtering, grouping and sorting, and its joins are one to two orders of magnitude faster than `merge()`; grouped aggregation is far faster than `aggregate()`. For a *single-column lookup* the vectorized base-R `match()` is even faster than a data.table join, because it only builds one integer index – a good example that a well-chosen vectorized primitive is hard to beat, whereas data.table's join is the general tool for multi-column / many-to-many joins. For row-level arithmetic, vectorized base R and data.table are equivalent and both are thousands of times faster than `apply()` / `lapply()` / `map()`.
- **Memory:** `merge()` and `aggregate()` create several intermediate copies; data.table joins and especially the **update join** `:=` add a column in place, so the peak extra memory is only the size of the new column. `apply()` needs an extra matrix copy of the data.
- **Scalability:** all approaches grow roughly linearly with the number of rows, but with very different slopes – at 20 million rows `aggregate()` needs many seconds while `data.table` stays well below a second, so the gap widens as the data grows.
- **Programming complexity / readability:** vectorized code and data.table one-liners are both short and readable. `purrr::map_*()` is the most explicit and type-safe functional style but is slow when used per row. Parallel code adds the most complexity (cluster set-up, exporting functions/packages, seeds, cleanup) and only pays off for heavy, independent computations.

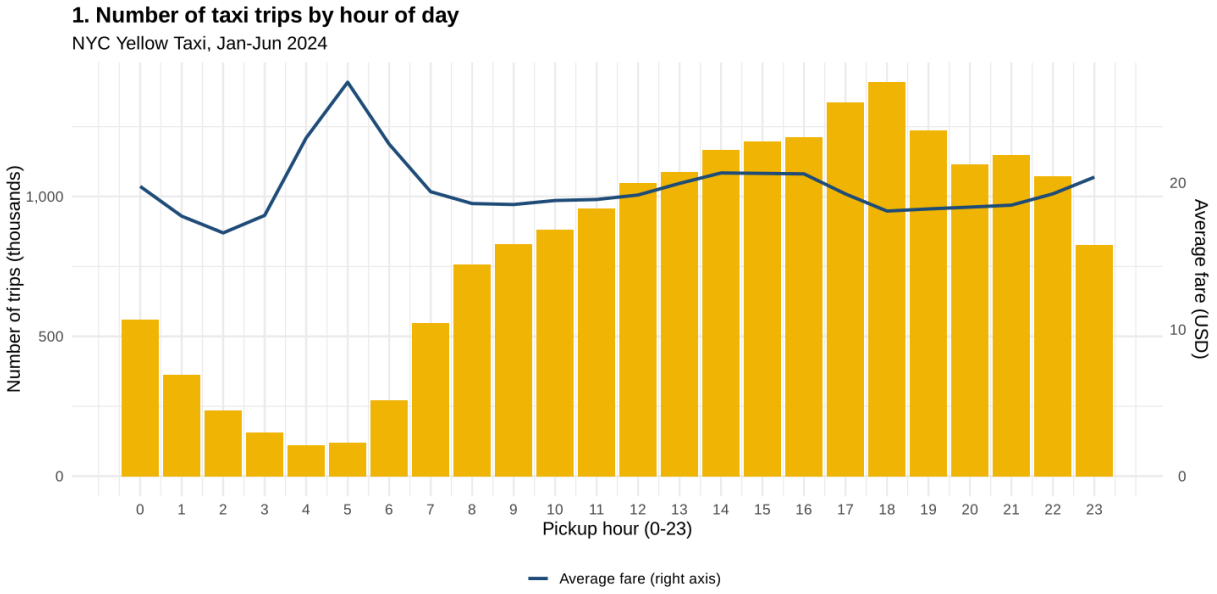
---

# 8. Task 6 – Data Visualization (ggplot2)

All figures are also saved as PNG files in `outputs/figures/`.

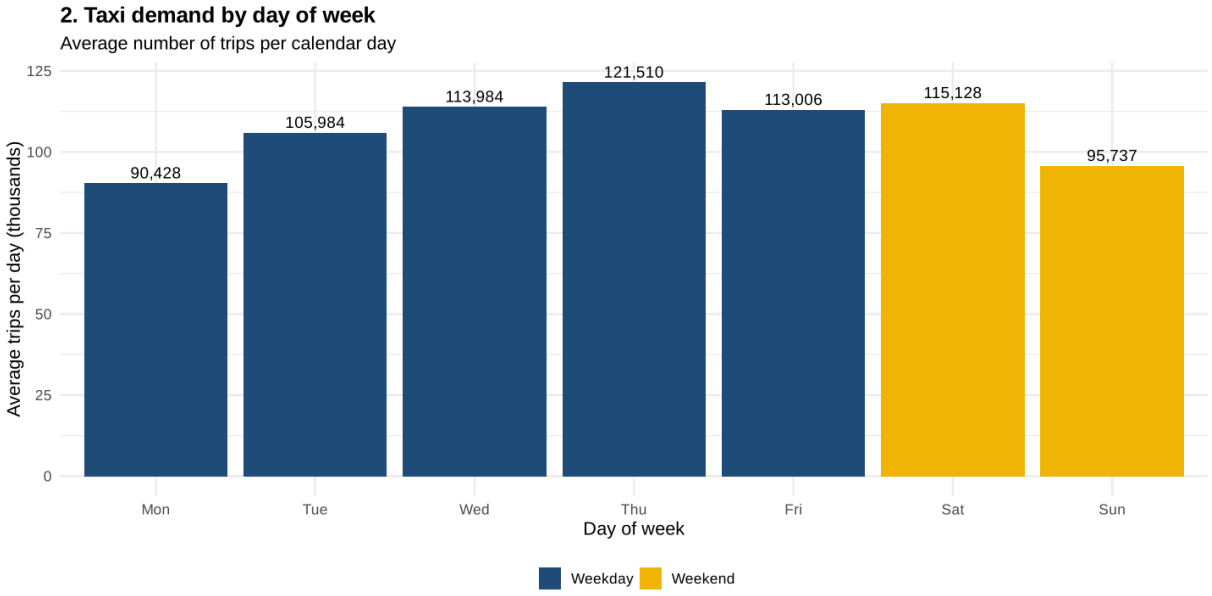
In [42]:
```
p <- ggplot(hourly, aes(pickup_hour, trips / 1e3)) +
  geom_col(fill = "#F2B705") +
  geom_line(aes(y = avg_fare * max(trips / 1e3) / max(avg_fare), colour =
```

```
"Average fare (right axis)"), linewidth = 1.1, group = 1) +
  scale_colour_manual(values = "#1F4E79", name = NULL) +
  scale_x_continuous(breaks = 0:23) +
  scale_y_continuous(labels = comma, sec.axis = sec_axis(~ . *
max(hourly$avg_fare) / max(hourly$trips / 1e3), name = "Average fare
(USD)")) +
  labs(title = "1. Number of taxi trips by hour of day", subtitle = "NYC
Yellow Taxi, Jan-Jun 2024",
       x = "Pickup hour (0-23)", y = "Number of trips (thousands)")
save_plot(p, "01_trips_by_hour")
```

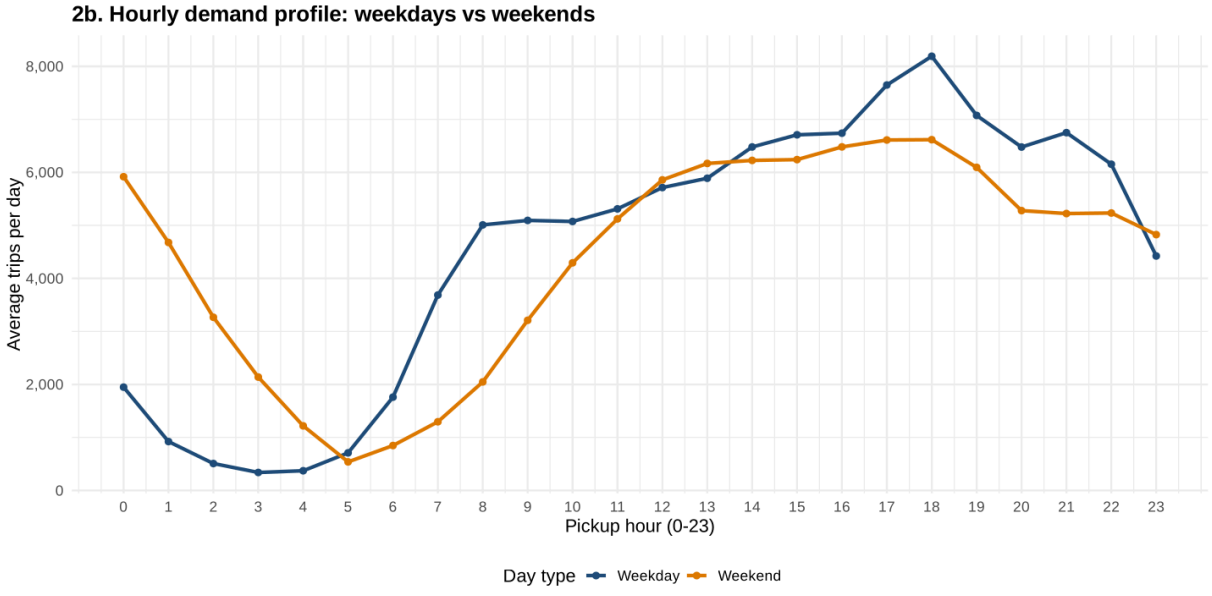



*Demand is lowest at 4–5 AM and peaks at 17–18 h; average fares are highest in the early morning when trips are longer.*

In [43]:
```
p <- ggplot(dow, aes(pickup_wday, avg_trips_per_day / 1e3, fill =
pickup_wday %in% c("Sat", "Sun"))) +
  geom_col() +
  geom_text(aes(label = comma(avg_trips_per_day)), vjust = -0.4, size = 4)
+
  scale_fill_manual(values = c(`FALSE` = "#1F4E79", `TRUE` = "#F2B705"),
labels = c("Weekday", "Weekend"), name = NULL) +
  labs(title = "2. Taxi demand by day of week", subtitle = "Average number
of trips per calendar day",
       x = "Day of week", y = "Average trips per day (thousands)")
save_plot(p, "02_demand_by_weekday")
```



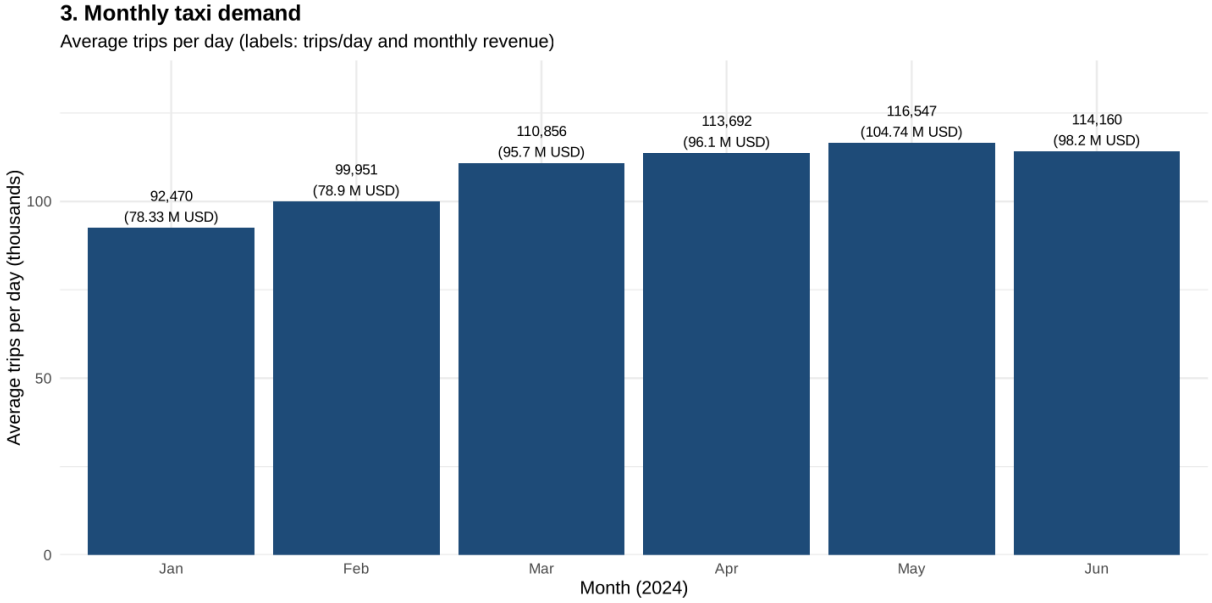

In [44]:
```
wk <- melt(dcast(dow_hour, pickup_hour ~ ifelse(is_weekend, "Weekend",
"Weekday"), value.var = "trips_per_day"),
           id.vars = "pickup_hour", variable.name = "day_type", value.name
= "trips_per_day")
p <- ggplot(wk, aes(pickup_hour, trips_per_day, colour = day_type)) +
geom_line(linewidth = 1.2) + geom_point() +
  scale_colour_manual(values = c(Weekday = "#1F4E79", Weekend =
"#E07A00"), name = "Day type") +
  scale_x_continuous(breaks = 0:23) + scale_y_continuous(labels = comma) +
  labs(title = "2b. Hourly demand profile: weekdays vs weekends", x =
"Pickup hour (0-23)", y = "Average trips per day")
save_plot(p, "02b_weekday_vs_weekend_hourly")
```




*Weekdays show morning and evening commuter peaks; weekends have no morning peak but much higher late-night demand (0–3 h).*

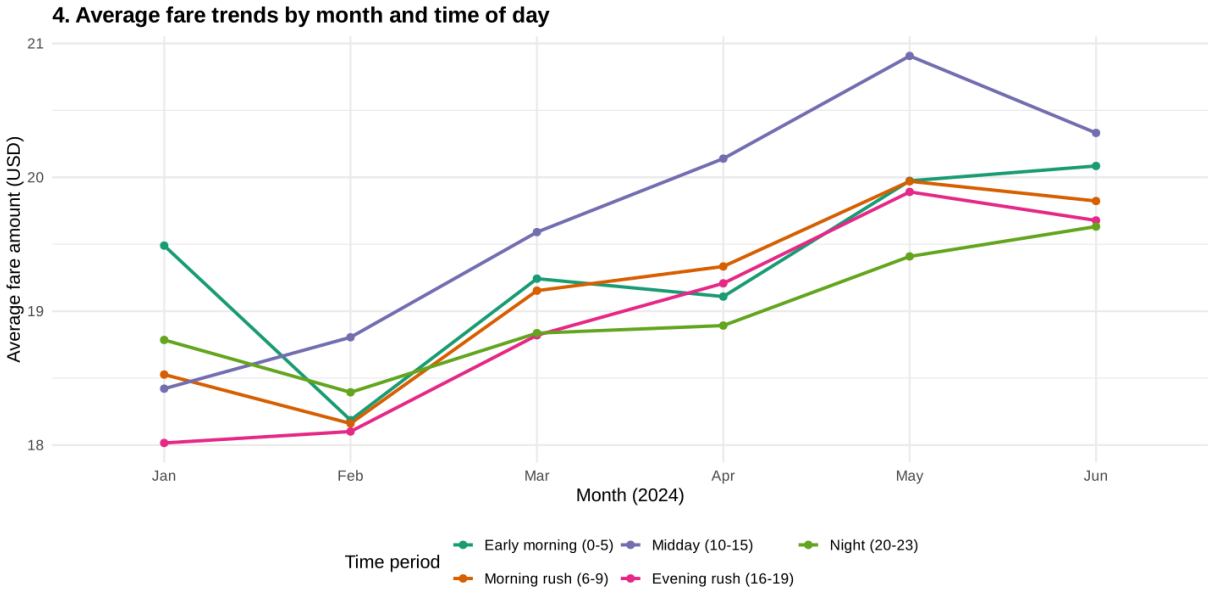
In [45]:
```
p <- ggplot(monthly, aes(month, trips_per_day / 1e3)) +
  geom_col(fill = "#1F4E79") +
  geom_text(aes(label = paste0(comma(trips_per_day), "\n(", revenue_musd,
" M USD)")), vjust = -0.2, size = 3.6) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.2))) +
  labs(title = "3. Monthly taxi demand", subtitle = "Average trips per day
```

```
(labels: trips/day and monthly revenue)",
       x = "Month (2024)", y = "Average trips per day (thousands)")
save_plot(p, "03_monthly_demand")
```



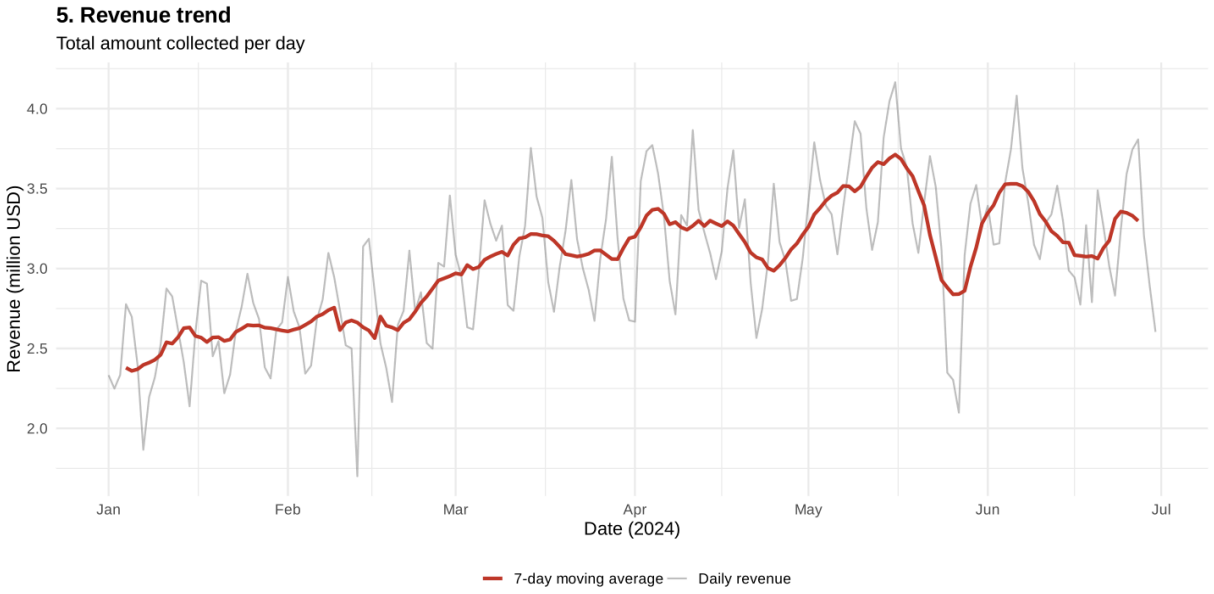

*January and February are the quietest months; demand peaks in spring (March–May).*

In [46]:
```
fare_trend <- trips[, .(avg_fare = mean(fare_amount), avg_total =
mean(total_amount)), by = .(month, time_period)]
p <- ggplot(fare_trend, aes(month, avg_fare, colour = time_period, group =
time_period)) +
  geom_line(linewidth = 1.1) + geom_point(size = 2) +
  scale_colour_brewer(palette = "Dark2", name = "Time period") +
  labs(title = "4. Average fare trends by month and time of day", x =
"Month (2024)", y = "Average fare amount (USD)") +
  guides(colour = guide_legend(nrow = 2))
save_plot(p, "04_average_fare_trends")
```



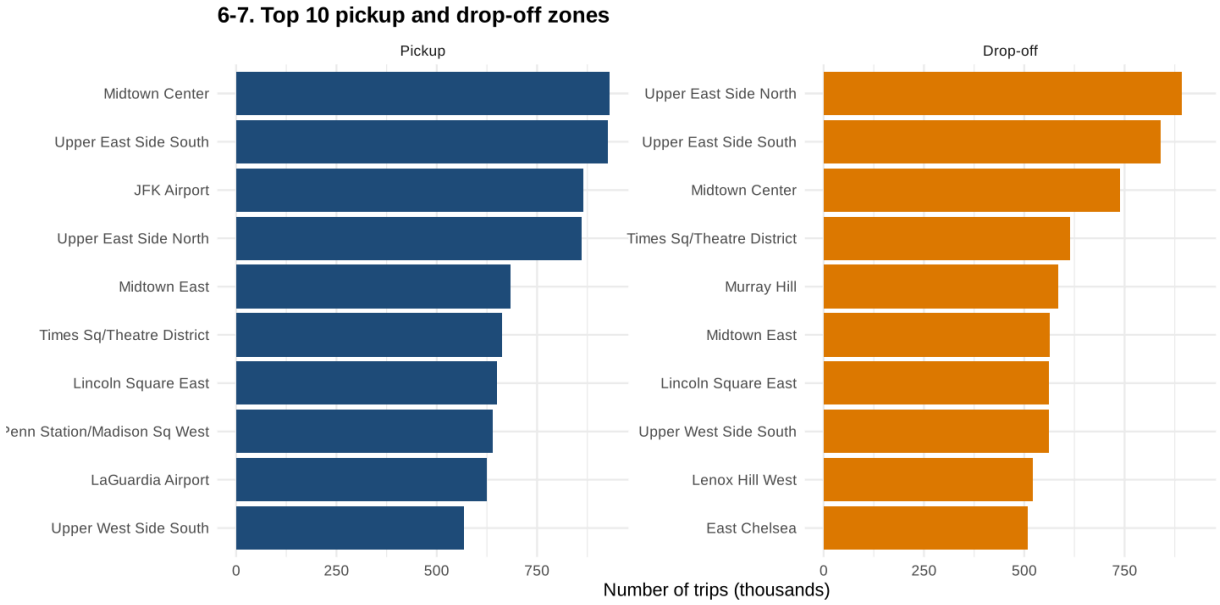

*Early-morning trips are consistently the most expensive on average; fares are stable across months.*

In [47]:
```
daily[, rev_7d := frollmean(revenue, 7, align = "center")]
p <- ggplot(daily, aes(as.Date(pickup_date))) +
  geom_line(aes(y = revenue / 1e6, colour = "Daily revenue"), alpha = 0.5)
+
  geom_line(aes(y = rev_7d / 1e6, colour = "7-day moving average"),
linewidth = 1.2, na.rm = TRUE) +
  scale_colour_manual(values = c(`Daily revenue` = "grey50", `7-day moving
average` = "#C0392B"), name = NULL) +
  scale_x_date(date_breaks = "1 month", date_labels = "%b") +
  labs(title = "5. Revenue trend", subtitle = "Total amount collected per
day", x = "Date (2024)", y = "Revenue (million USD)")
save_plot(p, "05_revenue_trend")
```



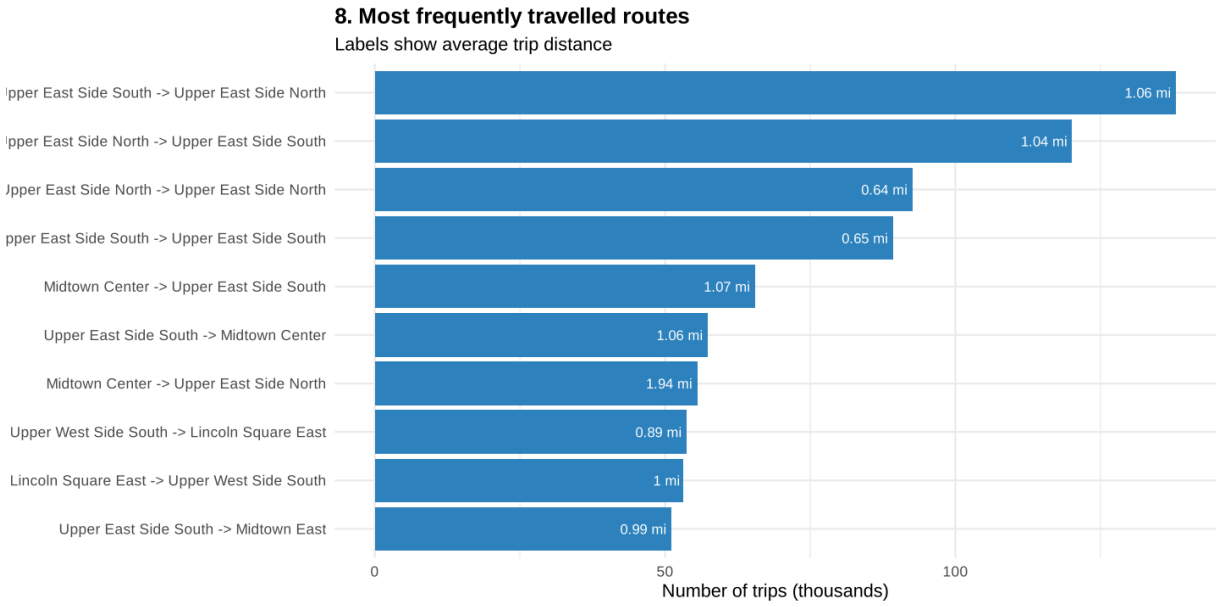

*Revenue shows a strong weekly cycle and dips on holidays (e.g. New Year, Memorial Day) and stormy days.*

In [48]:
```
top_pu <- trips[, .N, by = pu_zone][order(-N)][1:10]
top_do <- trips[, .N, by = do_zone][order(-N)][1:10]
top_zones <- rbind(top_pu[, .(zone = pu_zone, N, type = "Pickup")],
top_do[, .(zone = do_zone, N, type = "Drop-off")])
top_zones[, zone_in_panel := reorder(paste(zone, type, sep = "___"), N)]
# sort bars separately in each panel
p <- ggplot(top_zones, aes(zone_in_panel, N / 1e3, fill = type)) +
geom_col(show.legend = FALSE) + coord_flip() +
  facet_wrap(~ factor(type, c("Pickup", "Drop-off")), scales = "free_y") +
  scale_x_discrete(labels = function(x) sub("___.*$", "", x)) +
  scale_fill_manual(values = c(Pickup = "#1F4E79", `Drop-off` =
"#E07A00")) +
  labs(title = "6-7. Top 10 pickup and drop-off zones", x = NULL, y =
"Number of trips (thousands)")
save_plot(p, "06_07_top_pickup_dropoff_zones", width = 13)
```

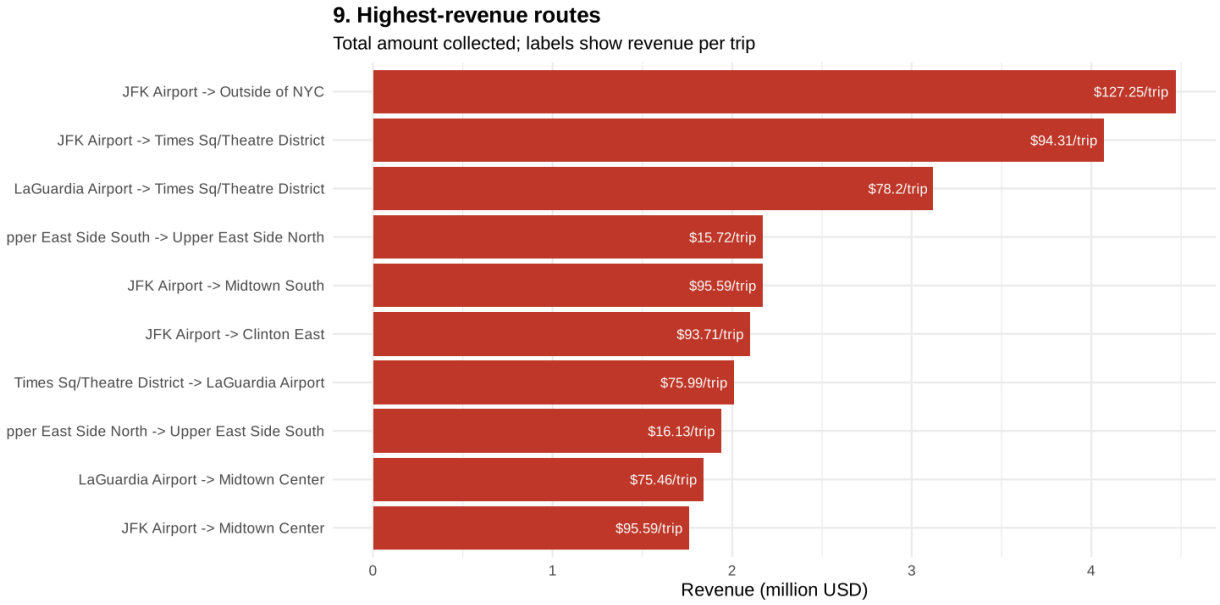



*Midtown/Upper East Side and JFK dominate pickups; drop-offs are spread over residential Manhattan zones.*

In [49]:
```
tr <- copy(top_routes[1:10])[, route := paste(pu_zone, "->", do_zone)]
p <- ggplot(tr, aes(reorder(route, trips), trips / 1e3)) + geom_col(fill =
"#2E86C1") + coord_flip() +
  geom_text(aes(label = paste0(avg_distance, " mi")), hjust = 1.1, colour
= "white", size = 3.5) +
  labs(title = "8. Most frequently travelled routes", subtitle = "Labels
show average trip distance",
       x = NULL, y = "Number of trips (thousands)")
save_plot(p, "08_top_routes", width = 12)
```

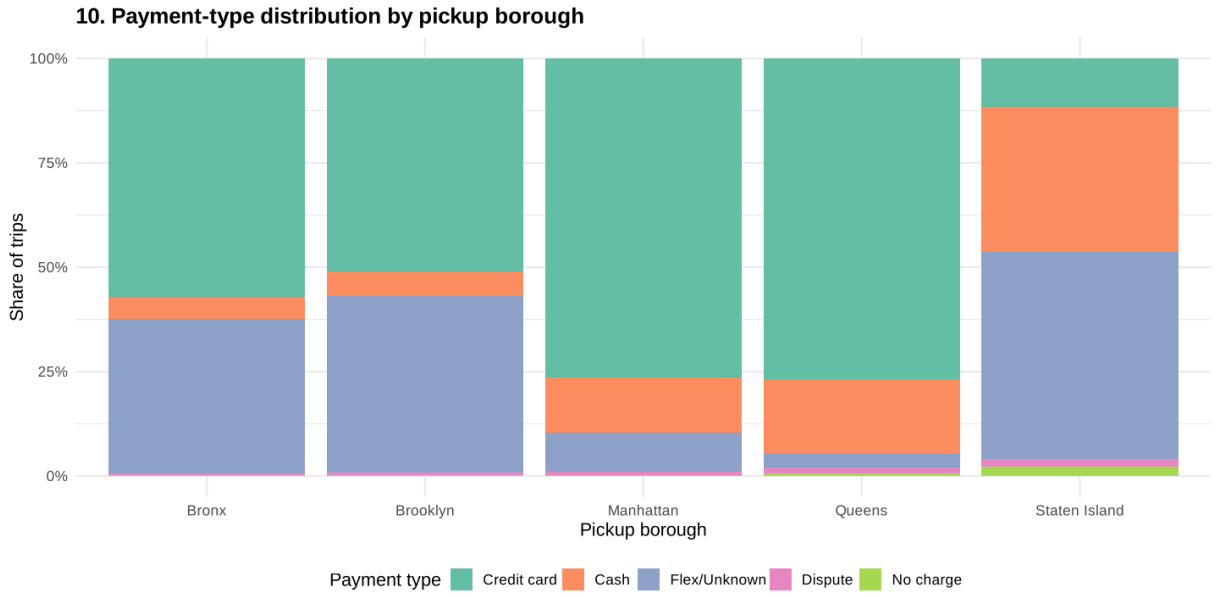



In [50]:
```
rr <- copy(top_rev_routes[1:10])[, route := paste(pu_zone, "->", do_zone)]
p <- ggplot(rr, aes(reorder(route, revenue_musd), revenue_musd)) +
geom_col(fill = "#C0392B") + coord_flip() +
  geom_text(aes(label = paste0("$", revenue_per_trip, "/trip")), hjust =
1.1, colour = "white", size = 3.5) +
  labs(title = "9. Highest-revenue routes", subtitle = "Total amount
collected; labels show revenue per trip",
       x = NULL, y = "Revenue (million USD)")
save_plot(p, "09_top_revenue_routes", width = 12)
```




*Short Upper-East-Side hops are the most frequent routes, but the money is made on airport routes, which earn several times more per trip.*

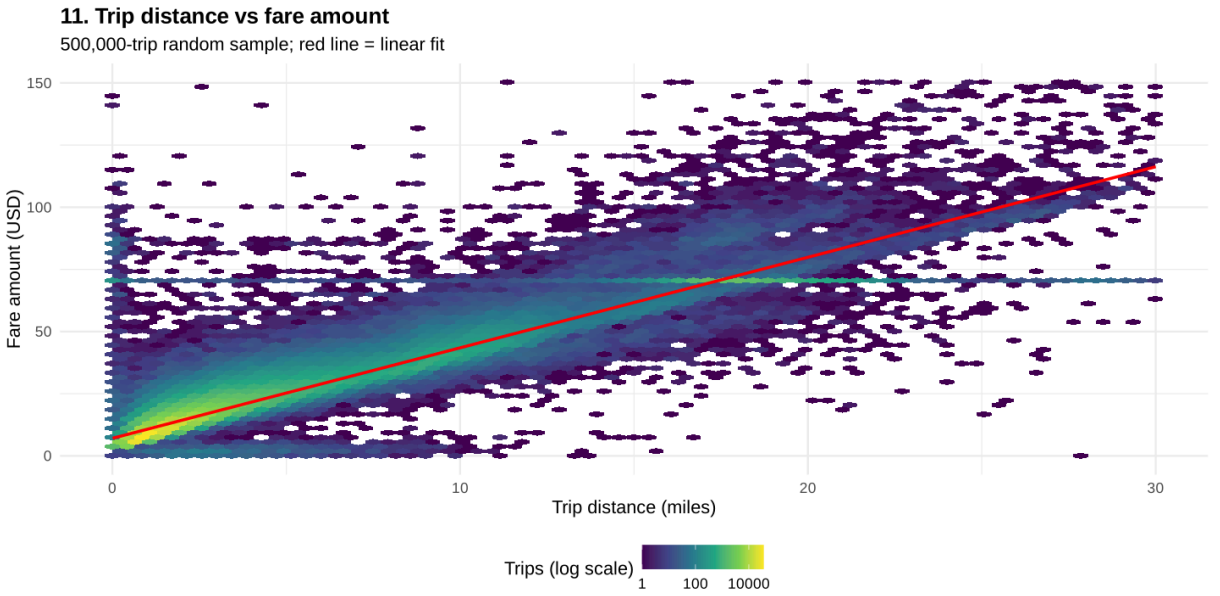
In [51]:
```
pb <- pay_borough[pu_borough != "EWR"]
p <- ggplot(pb, aes(pu_borough, share_pct, fill = payment_type)) +
geom_col(position = "fill") +
  scale_y_continuous(labels = percent) + scale_fill_brewer(palette =
"Set2", name = "Payment type") +
  labs(title = "10. Payment-type distribution by pickup borough", x =
"Pickup borough", y = "Share of trips")
save_plot(p, "10_payment_by_borough")
```




*Credit cards dominate everywhere, but cash and flex/unknown payments are more common outside Manhattan.*

In [52]:
```
hex_samp <- trips[sample(.N, 5e5)][trip_distance <= 30 & fare_amount <=
150]
p <- ggplot(hex_samp, aes(trip_distance, fare_amount)) +
  geom_hex(bins = 70) + scale_fill_viridis_c(trans = "log10", name =
"Trips (log scale)") +
  geom_smooth(method = "lm", formula = y ~ x, colour = "red", linewidth =
```

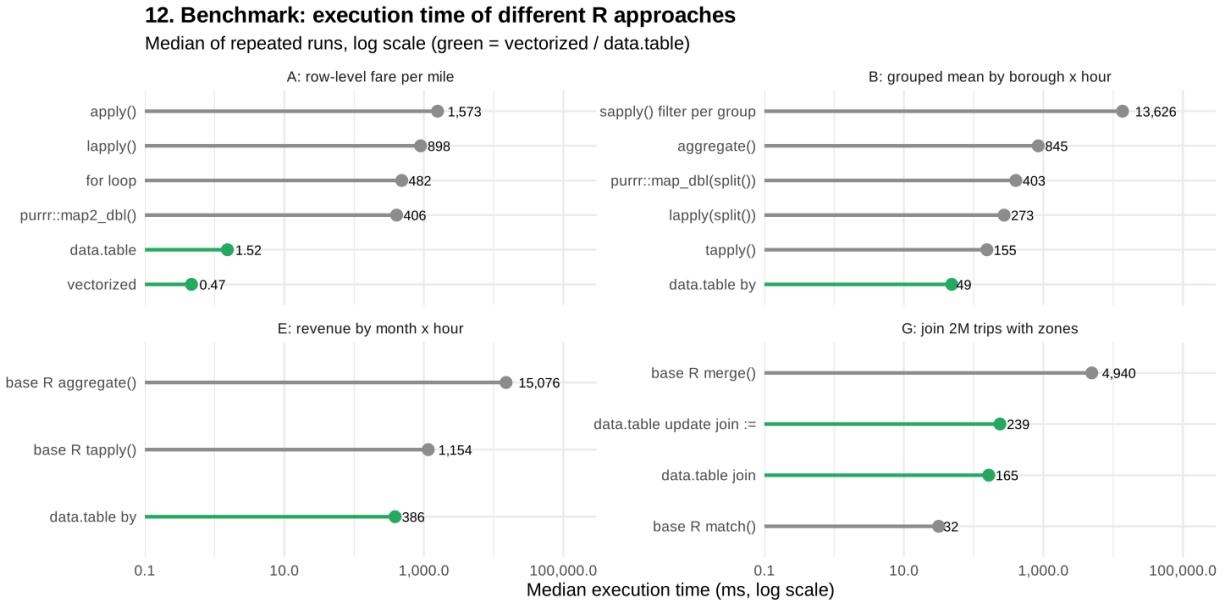
```
1) +
  labs(title = "11. Trip distance vs fare amount", subtitle = "500,000-
trip random sample; red line = linear fit",
       x = "Trip distance (miles)", y = "Fare amount (USD)")
save_plot(p, "11_distance_vs_fare")
```



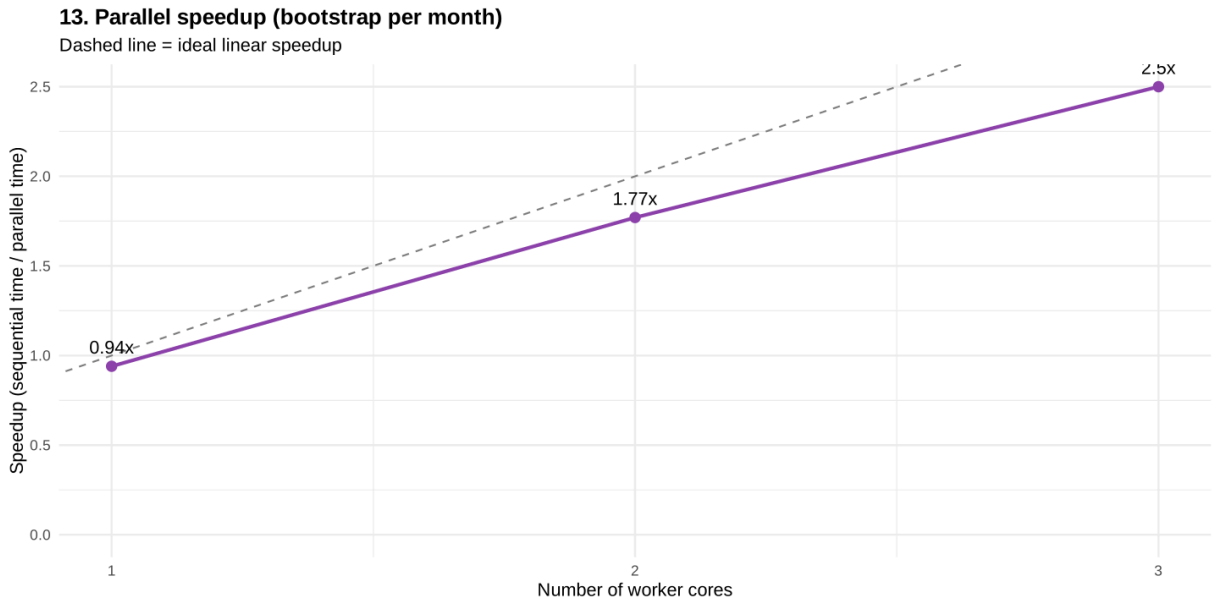

*Fare rises almost linearly with distance. The horizontal band at ≈ 70 USD is the JFK–Manhattan flat fare.*

In [53]:

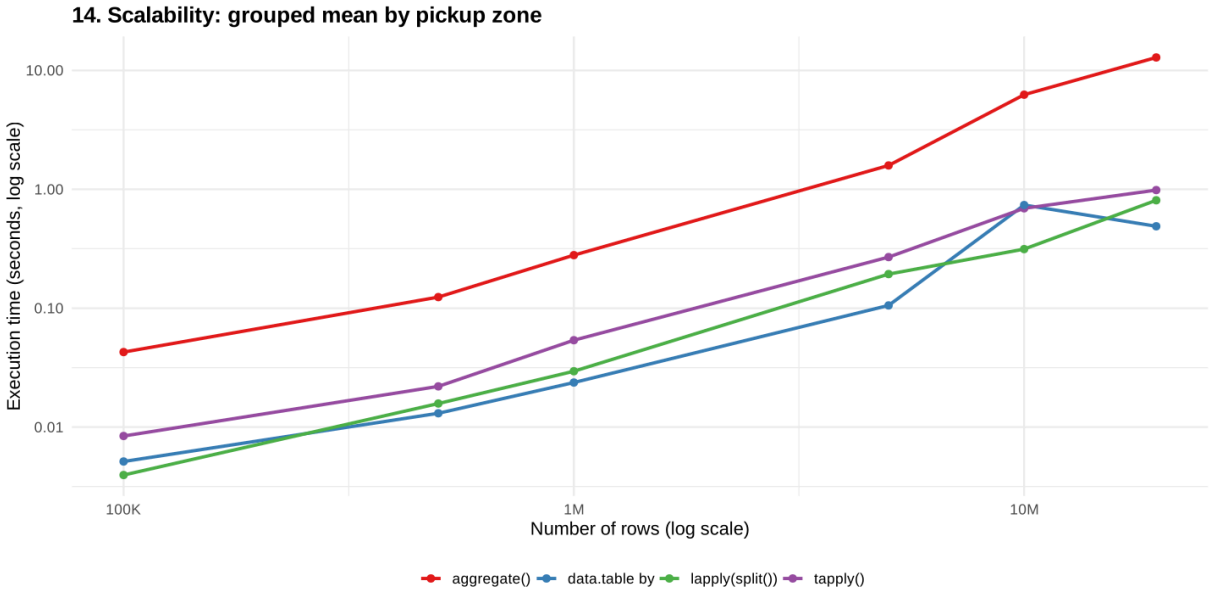
```
bp <- bench_all[comparison %in% c("A: row-level fare per mile", "B:
grouped mean by borough x hour", "E: revenue by month x hour", "G: join 2M
trips with zones")]
bp[, fast_style := grepl("data.table|vectorized", approach)]
bp[, approach_in_panel := reorder(paste(approach, comparison, sep =
"___"), median_ms)]   # sort within each panel
p <- ggplot(bp, aes(approach_in_panel, median_ms, colour = fast_style)) +
  geom_segment(aes(xend = approach_in_panel, y = 0.1, yend = median_ms),
linewidth = 1.2, show.legend = FALSE) +
  geom_point(size = 3.5, show.legend = FALSE) +
  geom_text(aes(label = ifelse(median_ms >= 10, comma(median_ms, accuracy
= 1), round(median_ms, 2))), hjust = -0.3, size = 3.4, colour = "black") +
  coord_flip() + facet_wrap(~ comparison, scales = "free_y", ncol = 2) +
  scale_x_discrete(labels = function(x) sub("___.*$", "", x)) +
  scale_y_log10(labels = comma, expand = expansion(mult = c(0, 0.25))) +
  scale_colour_manual(values = c(`FALSE` = "grey55", `TRUE` = "#27AE60"))
+
  labs(title = "12. Benchmark: execution time of different R approaches",
subtitle = "Median of repeated runs, log scale (green = vectorized /
data.table)",
       x = NULL, y = "Median execution time (ms, log scale)")
save_plot(p, "12_benchmark_comparison", width = 14, height = 8)
```




In [54]:
```
p <- ggplot(scaling, aes(cores, speedup)) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour =
"grey50") +
  geom_line(colour = "#8E44AD", linewidth = 1.2) + geom_point(size = 3,
colour = "#8E44AD") +
  geom_text(aes(label = paste0(speedup, "x")), vjust = -1) +
  scale_x_continuous(breaks = scaling$cores) + expand_limits(y = 0) +
  labs(title = "13. Parallel speedup (bootstrap per month)", subtitle =
"Dashed line = ideal linear speedup",
       x = "Number of worker cores", y = "Speedup (sequential time /
parallel time)")
save_plot(p, "13_parallel_speedup", width = 9)
```




In [55]:
```
p <- ggplot(scal, aes(rows, median_ms / 1000, colour = approach)) +
geom_line(linewidth = 1.1) + geom_point(size = 2) +
  scale_x_log10(labels = label_number(scale_cut = cut_short_scale())) +
scale_y_log10() +
  scale_colour_brewer(palette = "Set1", name = NULL) +
  labs(title = "14. Scalability: grouped mean by pickup zone", x = "Number
of rows (log scale)", y = "Execution time (seconds, log scale)")
save_plot(p, "14_scalability")
```




*data.table is fastest for every size and the gap widens as the number of rows grows; parallel speedup is real but sub-linear.*

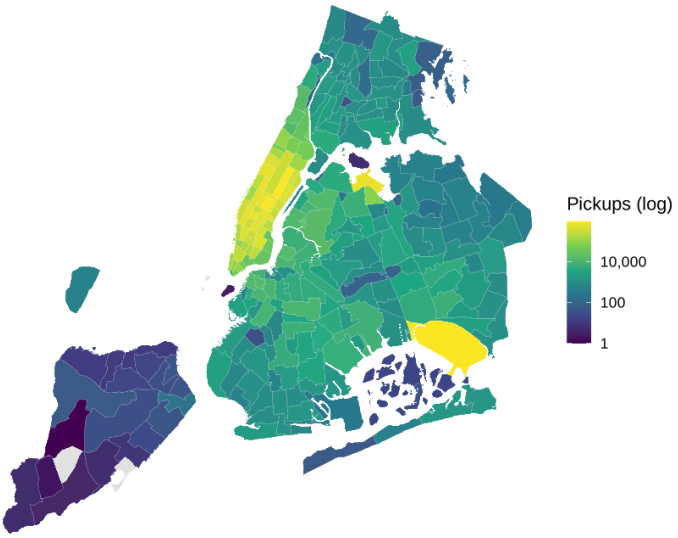
## 8.1 Optional advanced task – geographical visualizations

Zone polygons from the official TLC shapefile are used for a static choropleth (ggplot2 + sf) and an interactive **leaflet** map of the top pickup zones.

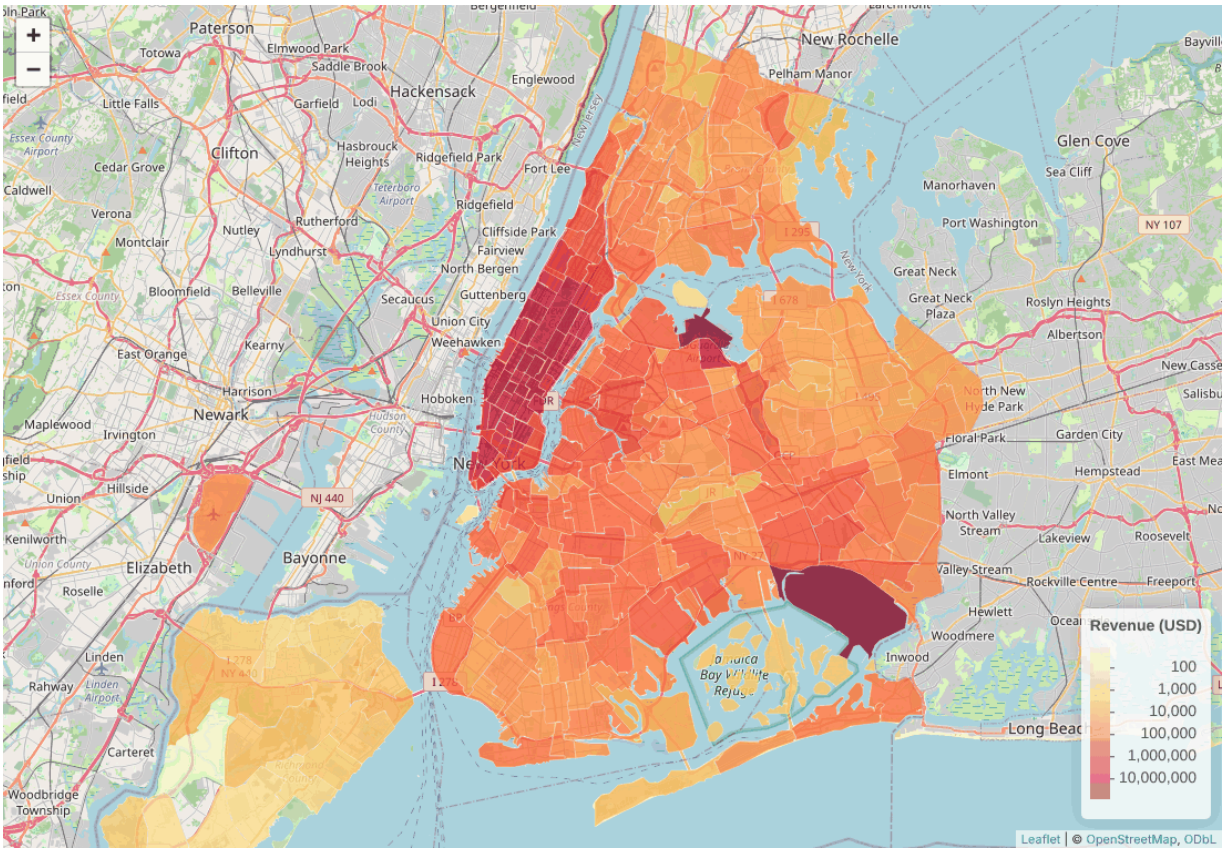
In [56]:
```
if (!HAVE_MAPS) {
  message("sf / leaflet are not available - skipping the optional map
section.")
} else {
suppressPackageStartupMessages({ library(sf); library(leaflet);
library(htmlwidgets) })
shape_dir <- file.path(DATA_DIR, "taxi_zones")
if (!dir.exists(shape_dir)) unzip(shape_zip, exdir = shape_dir)
shp <- list.files(shape_dir, pattern = "\\.shp$", recursive = TRUE,
full.names = TRUE)[1]
zones_sf <- st_transform(st_make_valid(st_read(shp, quiet = TRUE)), 4326)
zone_stats <- trips[, .(pickups = .N, revenue = sum(total_amount),
avg_total = mean(total_amount)), by = .(LocationID = PULocationID)]
zones_sf <- merge(zones_sf, zone_stats, by = "LocationID", all.x = TRUE)

p <- ggplot(zones_sf) + geom_sf(aes(fill = pickups), colour = "white",
linewidth = 0.05) +
  scale_fill_viridis_c(trans = "log10", labels = comma, name = "Pickups
(log)", na.value = "grey90") +
  labs(title = "15. Pickups by taxi zone") +
  theme_void(base_size = 13) + theme(legend.position = "right", plot.title
= element_text(face = "bold"))
save_plot(p, "15_pickup_map", width = 9, height = 9)
}
```

**15. Pickups by taxi zone**




In [57]:
```
if (HAVE_MAPS) {
# an interactive choropleth: zone polygons never overlap, unlike circle
markers
map_sf <- zones_sf[!is.na(zones_sf$revenue), ]
pal <- colorNumeric("YlOrRd", domain = log10(map_sf$revenue))
m <- leaflet(map_sf) |>
  addTiles() |>                                    # plain OpenStreetMap
tiles, no API key needed
  setView(lng = -73.93, lat = 40.73, zoom = 11) |>
  addPolygons(fillColor = ~pal(log10(revenue)), fillOpacity = 0.75, weight
= 0.6, color = "white",
              highlightOptions = highlightOptions(weight = 2.5, color =
"#1F4E79", bringToFront = TRUE),
              label = ~sprintf("%s (%s): %s pickups, $%s M revenue, $%.2f
per trip",
                               zone, borough, comma(pickups),
round(revenue / 1e6, 2), avg_total)) |>
  addLegend("bottomright", pal = pal, values = ~log10(revenue), title =
"Revenue (USD)",
            labFormat = labelFormat(transform = function(x) round(10^x)))
saveWidget(m, file.path(normalizePath(OUT_DIR),
"leaflet_revenue_by_zone.html"), selfcontained = TRUE)
m
}
```




*Each taxi zone is shaded by the **total revenue collected from trips starting there** (log scale). Midtown, the Upper East Side and the two airports are the dark, high-revenue areas; hover over any zone to see its pickups, revenue and average amount per trip.*

> **Note on the saved notebook:** an interactive leaflet widget embeds the whole map library (about 4 MB), which stops GitHub from rendering the notebook in the browser. The committed notebook therefore shows a static preview of this map. Run the cell yourself, or open `outputs/leaflet_revenue_by_zone.html`, for the fully interactive version.

---

## 9. Observations

The key measured numbers from this run are collected below (they will vary slightly between machines).

In [58]:
```
bm <- function(cmp, appr) bench_all[comparison == cmp & approach == appr,
median_ms]
obs <- data.table(
  metric = c("Trips analysed (after cleaning)", "Peak pickup hour",
"Busiest weekday (avg trips/day)", "Busiest month (avg trips/day)",
             "Top revenue route", "Credit-card share of trips (%)",
"Distance-fare correlation",
             "Row-level: apply() vs vectorized (x slower)", "Grouping 2M:
lapply(split()) vs data.table (x slower)",
             "Grouping 20M: aggregate() vs data.table (x slower)", "Join
2M: merge() vs data.table (x slower)",
```

```
             sprintf("Parallel speedup with %d cores", N_CORES), "Light
task: parallel vs sequential (speedup)"),
  value = c(format(nrow(trips), big.mark = ","),
paste0(hourly[which.max(trips), pickup_hour], ":00"),
           as.character(dow[which.max(avg_trips_per_day), pickup_wday]),
as.character(monthly[which.max(trips_per_day), month]),
           top_rev_routes[1, paste(pu_zone, "->", do_zone)],
pay_overall[payment_type == "Credit card", share_pct],
           round(trips[, cor(trip_distance, fare_amount)], 3),
           round(bm("A: row-level fare per mile", "apply()") / bm("A:
row-level fare per mile", "vectorized")),
           round(bm("B: grouped mean by borough x hour",
"lapply(split())") / bm("B: grouped mean by borough x hour", "data.table
by")),
           round(bm("E: revenue by month x hour", "base R aggregate()") /
bm("E: revenue by month x hour", "data.table by")),
           round(bm("G: join 2M trips with zones", "base R merge()") /
bm("G: join 2M trips with zones", "data.table join")),
           paste0(speedup_main, "x"), paste0(light[2, speedup_vs_seq],
"x")))
obs
```

A data.table: 13 × 2

| metric | value |
|---|---|
| <chr> | <chr> |
| Trips analysed (after cleaning) | 19,650,216 |
| Peak pickup hour | 18:00 |
| Busiest weekday (avg trips/day) | Thu |
| Busiest month (avg trips/day) | May |
| Top revenue route | JFK Airport → Outside of NYC |
| Credit-card share of trips (%) | 76.2 |
| Distance-fare correlation | 0.936 |
| Row-level: apply() vs vectorized (x slower) | 3382 |
| Grouping 2M: lapply(split()) vs data.table (x slower) | 6 |
| Grouping 20M: aggregate() vs data.table (x slower) | 39 |
| Join 2M: merge() vs data.table (x slower) | 30 |
| Parallel speedup with 3 cores | 2.11x |
| Light task: parallel vs sequential (speedup) | 0.01x |

1. **Demand** is strongly time dependent: an evening peak around 17–18 h, a minimum at 4–5 AM, the highest per-day demand on mid-week days and the lowest on Sundays, and a spring high after the winter months.
2. **Revenue** is concentrated in the midday and evening-rush periods, while **airport routes** (JFK, LaGuardia) generate by far the highest revenue per trip and dominate the highest-revenue routes.

3. **Fare vs distance** is strongly linear, but the fare per mile is highest for very short trips.
4. **Payments:** credit cards account for about three quarters of trips; cash and unrecorded ("flex/unknown") payments are relatively more common outside Manhattan, and tips are recorded almost only for card payments.
5. **Performance:** data.table and vectorized operations were faster than functional row-wise code by orders of magnitude; data.table beat base R `aggregate()` / `merge()` by a large factor while using less memory; parallel processing gave a real but sub-linear speedup for the heavy bootstrap and was slower for the light task.

---

# 10. Task 7 – Critical Analysis and Recommendation

**1. Which approach provides the best execution performance? data.table** for data manipulation (filter, group, sort, join) and **vectorized** code for element-wise arithmetic. In the benchmarks data.table was the fastest entry for filtering, grouping and sorting, many times faster than base R `aggregate()` and `merge()`, and vectorized arithmetic was thousands of times faster than row-wise `apply()`. The exceptions are instructive: a single-column `match()` lookup beat the data.table join, and functional `lapply()` over a few columns matched data.table – iterating over a few large units costs nothing.

**2. How does data.table improve the processing of large datasets?** (a) Operations are implemented in optimised, **multi-threaded C** (radix sorting and grouping, GForce for `mean` / `sum`); (b) `:=`, `set()`, `setkey()`, `setorder()` and update joins modify data **by reference**, so a 20-million-row table is never copied – visible in the much lower peak memory of the update join; (c) `fread()` / `setDT()` give fast import; (d) the concise `DT[i, j, by]` syntax combines filtering, computing and grouping in one optimised step.

**3. When is functional programming preferable?** When iterating over a **small number of large items** – columns, files, months, model specifications, groups – where the per-call overhead is negligible (Experiment C showed `lapply()` / `map_dbl()` as fast as `colMeans()`), and when clarity and composability matter (purrr's type-stable `map_dbl()`, pipelines, applying the same function to every partition). It is also the natural interface to parallelism (`foreach`, `parLapply`). It should not be used to loop over millions of rows.

**4. When do vectorized operations provide better performance?** Whenever the same operation is applied to every element of a vector/column (arithmetic, comparisons, `ifelse`, `pmin`, cumulative functions): one call into compiled code replaces millions of interpreted R calls, giving the 1000×-scale gains measured in Experiment A.

**5. What performance improvement was obtained using parallel computing?** For the CPU-intensive month-wise bootstrap the measured speedup is shown in the scaling table (Section 6.5) and the observations table – a clear gain that increases with the number of cores but stays below the ideal (efficiency below 100 %).

**6. Does parallel computing always result in faster execution? No.** The lightweight task (mean fare per month) was *slower* in parallel than sequentially, because starting workers and serialising ≈ 600 MB of partitions costs much more than the computation. Even for the heavy task, overheads (data transfer, uneven number of partitions per worker, memory bandwidth) prevent linear speedup. A single multi-threaded data.table grouping was faster than both partitioned versions for the light task. Parallelism helps only when the computation per partition is large relative to the communication cost.

**7. Trade-offs between speed, memory, readability and scalability**

| Approach | Speed | Memory | Readability | Scalability |
|---|---|---|---|---|
| `apply()` / `lapply()` / `map()` per row | very slow | high (per-call allocation, matrix copy) | good | poor |
| `for` loop (pre-allocated) | very slow | low | good for beginners, verbose | poor |
| Vectorized base R | very fast | moderate (temporary vectors) | very good | good |
| Base R `aggregate()` / `merge()` | slow | high (copies) | good | moderate |
| data.table | fastest | lowest (by reference) | good once learnt, terse syntax | excellent |
| foreach + doParallel | faster only for heavy tasks | high (data copied to every worker) | more complex code | good for CPU-bound, independent tasks |

**8. Recommendation for large-scale analytical applications** Use **data.table as the core engine** (import with arrow/fread, clean, join and aggregate by reference), write element-wise logic in **vectorized** form, use **functional programming** ( `lapply` / `purrr::map` ) for iterating over files, columns or partitions, and add **foreach + doParallel** only for genuinely CPU-intensive, independent tasks such as bootstrapping, simulation or model fitting per partition – after checking with a benchmark that the speedup outweighs the overhead. This recommendation is directly supported by the benchmark and speedup tables obtained in this notebook.

---

# 11. Conclusion

An end-to-end, reproducible big-data workflow was built in R for ≈ 20 million NYC Yellow Taxi trips (January–June 2024). data.table made it possible to import, clean, enrich, join and aggregate the full dataset in seconds on a laptop with modest memory, by working in place. The analysis showed clear temporal demand patterns, the dominance of credit-card payments and the outsized revenue of airport routes. The benchmarks demonstrated that **how** R code is written matters more than the hardware: row-wise functional code was thousands of times slower than vectorized code, base R grouping and joining were many times slower and more memory hungry than data.table, and parallel computing with foreach/doParallel delivered a useful but sub-linear speedup only for compute-heavy work. The recommended architecture is therefore data.table + vectorization at the core, functional programming for iteration over a few large units, and parallelism only where benchmarks prove it worthwhile.

In [59]:
```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 24.04.5 LTS

Matrix products: default
BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0  LAPACK version 3.12.0

locale:
 [1] LC_CTYPE=C.UTF-8    LC_NUMERIC=C        LC_TIME=C           LC_COLLATE=C        LC_MONETARY=C
LC_MESSAGES=C
 [7] LC_PAPER=C          LC_NAME=C           LC_ADDRESS=C        LC_TELEPHONE=C      LC_MEASUREMENT=C
LC_IDENTIFICATION=C

time zone: Etc/UTC
tzcode source: system (glibc)

attached base packages:
[1] parallel  stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
 [1] htmlwidgets_1.6.4    leaflet_2.2.3        sf_1.1-3             scales_1.4.0         lubridate_1.9.5
purrr_1.2.2
 [7] microbenchmark_1.5.0 doParallel_1.0.17    iterators_1.0.14     foreach_1.5.2        ggplot2_4.0.3
arrow_26.0.0
[13] data.table_1.18.6.1

loaded via a namespace (and not attached):
 [1] generics_0.1.4     class_7.3-24       KernSmooth_2.23-27 lattice_0.23-1     digest_0.6.39
magrittr_2.0.5     evaluate_1.0.5
 [8] grid_4.6.1         timechange_0.4.0   RColorBrewer_1.1-3 pbdZMQ_0.3-14      fastmap_1.2.0
jsonlite_2.0.0     e1071_1.7-17
[15] DBI_1.3.0          crosstalk_1.2.2    viridisLite_0.4.3  jquerylib_0.1.4    codetools_0.2-20
cli_3.6.6          rlang_1.3.0
[22] crayon_1.5.3       units_1.0-1        bit64_4.8.6        yaml_2.3.12        base64enc_0.1-6
withr_3.0.3        repr_1.1.7
[29] tools_4.6.1        uuid_1.2-2         assertthat_0.2.1   IRdisplay_1.1      vctrs_0.7.3
R6_2.6.1           proxy_0.4-29
[36] lifecycle_1.0.5    classInt_0.4-11    bit_4.6.0          pillar_1.11.1      hexbin_1.28.6
gtable_0.3.6       glue_1.8.1
[43] Rcpp_1.1.2         xfun_0.62          tidyselect_1.2.1   knitr_1.52         IRkernel_1.3.2
farver_2.1.2       htmltools_0.5.9
[50] rmarkdown_2.32     labeling_0.4.3     compiler_4.6.1     S7_0.2.2
```